



Hardware Scheduling approach on FPGA's for optimising DSP based workflows

Connor Sinclair

This thesis is presented as part of the requirements for the conferral of the degree:

Bachelor of Software Engineering (Hons.)

Supervisor:
Dr. Philip Leong

The University of Sydney
School of Engineering

2025

Declaration

I, *Connor Sinclair*, declare that this thesis is submitted in partial fulfilment of the requirements for the conferral of the degree *Bachelor of Software Engineering (Hons.)*, from the University of Sydney, is wholly my own work unless otherwise referenced or acknowledged. This document has not been submitted for qualifications at any other academic institution.

Connor Sinclair

October 23, 2025

Abstract

FPGA toolchains typically perform static compilation and mapping of fixed designs. In practice, audio signals and user-defined module graphs vary over time, presenting an opportunity to predictively reconfigure the FPGA fabric. This thesis proposes an operator-theoretic and analytic framework that models DSP modules as dynamical systems, using observables and new signal correlation metrics to forecast workload and adjust resource allocation. Unlike conventional flows, this approach can anticipate the evolving demands of a modular audio effect chain (including non-linear modules) and remap kernels (e.g. heavy multipliers or trigonometric units) onto LUTs, DSP slices, or BRAMs in accordance to resource demands. Expected contributions include: (1) an analytical model of audio DSP modules grounded in operator theory (Koopman embeddings) and state-space representations; (2) coherent metrics of signal interaction for resource tuning; and (3) a prototype architecture enabling runtime FPGA optimization in an accessible form. This work bridges a gap between control-theoretic modeling and FPGA architecture, offering audio engineers and embedded developers a more adaptive FPGA DSP platform that can improve throughput, support complex nonlinear & stochastic audio effects and enable massive real-time parallelisation.

Acknowledgements

Contents

Abstract	iii
Acknowledgements	iv
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Motivation	1
1.2 Aims	1
1.3 Contributions	1
1.4 Thesis Structure	1
2 Literature review	2
2.1 Dyadicization via Mahler (binomial) expansion on $\mathbb{Z}_2$	2
2.2 Walsh series: dyadic orthobasis and FPGA friendliness	3
3 Combined CSP & DSP approach to categorising signal families	4
3.1 Cyclostationary analysis	4
3.1.1 Relation between Strict Periodicity and Cyclostationarity . . .	5
3.2 Novel combined CSP, DSP approach to creating FPGA cache heirarchies	6
3.2.1 Group-Theoretic Interpretation	6
3.2.2 Link to Cyclostationary Signals	7
3.2.3 Procedure for storing arbitrary signals	8
3.3 Synthesizable construction of the recovery method, T , for FPGA's . .	8
3.3.1 Efficient FPGA Implementation of $\arccos(x)$ Using 'Quarter'- Domain Folding	8
3.3.2 Generalizing recovery method construction technique	10
3.3.3 Setup: Walsh basis and discrete (dyadic) synthesis	11
3.3.4 Forcing dyadic coefficients via dyadic sampling & quantization	11
3.3.5 Worked example: $f(x) = \sin(2\pi x)$	12
4 Architectural Design	13
4.1 Optimal Coefficient Quantization via Projection Methods	14
4.1.1 Chapter Objectives	14
4.1.2 Scope and Limitations	15

4.1.3	Problem Formulation	15
4.1.4	Projection Operator and Error Metric	16
4.1.5	Sensitivity Matrix Construction	17
4.1.6	Quantization Error Bounds	18
4.1.7	Optimal Bit Allocation Strategy	19
4.1.8	Evaluation of Projected Error	21
4.1.9	Projector Derivation for Sinc Function	23
4.2	Extending definition of the projector to accomodate generic $H(\omega)$. .	27
4.2.1	Dyadic Quantization of Projector Coefficients	29
4.3	Shannon Wavelet Basis and Orthonormal Extension	31
4.3.1	Shannon Wavelet Construction	32
4.3.2	Orthonormality and Parseval Identity	32
4.3.3	Projector Extension to the Wavelet Basis	33
4.3.4	Approximation of Arbitrary Bandlimited Functions	33
4.3.5	Error Analysis in the Wavelet Domain with Quantization . . .	34
4.3.6	Sensitivity Matrix for the Wavelet Basis with Quantization . .	34
4.3.7	Computational Advantages for Hardware Implementation . . .	35
4.3.8	Hardware-Efficient Kernel Selection with Quantization	36
4.4	Composite Kernel Architecture for Wavelet-Based Filtering	37
4.5	Adaptive Bit Allocation and Band-Limiting for Real-Time Signal Pro- cessing	37
5	Control theoretic model for dynamic FPGA resource balancing	39
A	Appendix	40
A.1	Auxiliary Lemmas	40
A.2	Hypergeometric Form for Real Arguments	41
A.3	Shannon Wavelet Derivations	42
A.3.1	Frequency Response Contribution $\Phi_n(\omega)$	42
A.3.2	Greedy Bit Allocation with Truncation-Quantization Trade-off	43
A.3.3	Hardware Resource Estimates for Shannon Wavelet Imple- mentation	44
A.3.4	Residue-Based Decomposition for Hardware Kernels	45
A.3.5	Signal Synthesis Capabilities of Shannon Wavelet Decomposition	46

List of Figures

4.1	Sinc remainder plot	27
4.2	Truncation error	28

List of Tables

4.1	Comparison of bit allocation methods.	20
A.1	Comparison of signal synthesis methods for bandlimited signals. . . .	49

Chapter 1

Introduction

1.1 Motivation

1.2 Aims

1.3 Contributions

1.4 Thesis Structure

Chapter 2

Literature review

2.1 Dyadicization via Mahler (binomial) expansion on $\mathbb{Z}_2$

Theorem 1 (Mahler). *For any continuous $f : \mathbb{Z}_2 \rightarrow \mathbb{Q}_2$, there exist unique coefficients $\{a_n\}_{n \geq 0} \subset \mathbb{Q}_2$ with $a_n \rightarrow 0$ in the 2-adic norm such that*

$$f(x) = \sum_{n=0}^{\infty} a_n \binom{x}{n}, \quad \binom{x}{n} = \frac{x(x-1) \cdots (x-n+1)}{n!},$$

and the series converges uniformly on $\mathbb{Z}_2$. Moreover $a_n = \Delta^n f(0)$ where Δ is the forward difference operator.

Proof (sketch). See standard expositions in p -adic analysis. The polynomials $\binom{x}{n}$ form an orthonormal basis of $C(\mathbb{Z}_2, \mathbb{Q}_2)$ under the sup norm, and Newton–Gregory interpolation yields the coefficients a_n . Uniform convergence follows from $a_n \rightarrow 0$ in $\mathbb{Q}_2$. $\square$

Corollary 1 (Dyadic evaluation and coefficient truncation). *Fix $m \geq 0$ and evaluate the Mahler truncation*

$$S_M(x) := \sum_{n=0}^M a_n \binom{x}{n}$$

on the dyadic grid $x = k/2^m$ ($0 \leq k \leq 2^m$). Then each $\binom{k/2^m}{n}$ is a rational with denominator a power of 2; hence if the a_n are chosen in $\mathbb{Q}$ with denominators a power of 2, every value $S_M(k/2^m)$ is a dyadic rational. Further, the truncation error satisfies

$$\|f - S_M\|_{\infty, \mathbb{Z}_2} \leq \sup_{n > M} |a_n|_2,$$

so dyadic accuracy is controlled directly by the 2-adic decay of a_n .

Remarks for hardware. (i) The basis $\binom{x}{n}$ is *integer-valued* on $x \in \mathbb{Z}$, which helps cancellation of odd primes in arithmetic paths. (ii) If f is originally real-analytic on $[-1, 1]$, one may transfer to $\mathbb{Z}_2$ by encoding inputs on a dyadic grid and using the same truncated binomial basis; the resulting polynomial has dyadic coefficients after rounding a_n to a fixed 2^{-b} grid, with an a priori sup-norm bound inherited from the Mahler tail.

2.2 Walsh series: dyadic orthobasis and FPGA friendliness

Let $\{r_j\}_{j \geq 0}$ be the Rademacher functions on $[0, 1)$, and define the Walsh functions (Paley ordering)

$$W_k(x) = \prod_{j=0}^{\infty} r_j(x)^{k_j}, \quad k = \sum_{j=0}^{\infty} k_j 2^j, \quad k_j \in \{0, 1\}.$$

Then $\{W_k\}_{k \geq 0}$ is a complete orthonormal basis of $L^2([0, 1))$. For $f \in L^2([0, 1))$ its Walsh–Fourier series is

$$f(x) \sim \sum_{k=0}^{\infty} \widehat{f}(k) W_k(x), \quad \widehat{f}(k) = \int_0^1 f(t) W_k(t) dt.$$

Dyadic rationality. If f is sampled on a dyadic grid of step 2^{-m} and the integrals are approximated by the exact rectangle rule on Walsh intervals, then each partial sum

$$S_N(x) = \sum_{k=0}^{N-1} \widehat{f}(k) W_k(x)$$

is a dyadic step function when the samples of f are dyadic rationals (the coefficients $\widehat{f}(k)$ become dyadic by construction). Computation uses the fast Walsh–Hadamard transform (FWHT), i.e. a butterfly network of adds/subtracts only.

Convergence and comparison to trigonometric series. Walsh series enjoy completeness and L^2 convergence analogous to Fourier series; uniform convergence holds under standard smoothness or bounded variation hypotheses adapted to dyadic intervals.

Chapter 3

Combined CSP & DSP approach to categorising signal families

3.1 Cyclostationary analysis

A signal with the property $x(t) = x(t + T) \forall t, T > 0$ is called *strictly periodic* (or *periodic pointwise*). A composition of periodic signals $h(t) = \sum_{i=0}^N x_i(t)$ is also periodic $\iff \frac{T_i}{T_j} \in \mathbb{Q} \forall i, j$ (i.e., the periods are commensurable). The period of $h(t)$ can therefore be described as

$$T = \text{lcm}(T_0, T_1, \dots, T_N).$$

A *wide-sense cyclostationary* (WSC) signal is defined by

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t + T)], \quad R_x(t, \tau) = R_x(t + T, \tau),$$

where $R_x(t, \tau) = \mathbb{E}[x(t)x^*(t + \tau)]$. That is, both the mean $\bar{x}(t)$ and the autocorrelation function $R_x(t, \tau)$ are periodic in t . More generally, a signal is cyclostationary *iff* there exists some n th-order nonlinear transformation of $x(t)$ that produces a finite additive sinusoidal component [**<empty citation>**].

The definition of the *spectral parameter* [**<empty citation>**] is

$$\hat{x}(\alpha) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t) e^{-j2\pi\alpha t} dt.$$

For $x(t) = a \cos(2\pi\beta t + \theta)$:

$$x(t) = \frac{a}{2} (e^{j(2\pi\beta t + \theta)} + e^{-j(2\pi\beta t + \theta)}).$$

Substituting into the definition:

$$\hat{x}(\alpha) = \lim_{T \rightarrow \infty} \frac{a}{2T} \left[e^{j\theta} \int_{-T/2}^{T/2} e^{j2\pi(\beta - \alpha)t} dt + e^{-j\theta} \int_{-T/2}^{T/2} e^{-j2\pi(\beta + \alpha)t} dt \right].$$

Recalling that

$$\int_{-T/2}^{T/2} e^{j2\pi f t} dt = \frac{\sin(\pi f T)}{\pi f},$$

we obtain

$$\hat{x}(\alpha) = \frac{a}{2} \lim_{T \rightarrow \infty} \frac{1}{T} \left[e^{j\theta} \frac{\sin(\pi(\beta - \alpha)T)}{\pi(\beta - \alpha)} + e^{-j\theta} \frac{\sin(\pi(\beta + \alpha)T)}{\pi(\beta + \alpha)} \right].$$

Equivalently,

$$\hat{x}(\alpha) = \frac{a}{2} \lim_{T \rightarrow \infty} [e^{j\theta} \text{sinc}((\beta - \alpha)T) + e^{-j\theta} \text{sinc}((\beta + \alpha)T)],$$

where $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$.

If $\alpha = \beta$:

$$\hat{x}(\alpha) = \frac{a}{2} e^{j\theta}.$$

If $\alpha = -\beta$:

$$\hat{x}(\alpha) = \frac{a}{2} e^{-j\theta}.$$

Otherwise:

$$\hat{x}(\alpha) = 0.$$

Thus:

$$\hat{x}(\alpha) = \begin{cases} \frac{a}{2} e^{j\theta}, & \alpha = \beta, \\ \frac{a}{2} e^{-j\theta}, & \alpha = -\beta, \\ 0, & \text{otherwise.} \end{cases}$$

This demonstrates that the time-averaged inner product survives only when the test frequency α matches a sinusoidal component β of the signal.

3.1.1 Relation between Strict Periodicity and Cyclostationarity

Every strictly periodic signal is trivially wide-sense cyclostationary. Indeed, if $x(t) = x(t + T)$, then

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t+T)], \quad R_x(t, \tau) = \mathbb{E}[x(t)x^*(t+\tau)] = \mathbb{E}[x(t+T)x^*(t+T+\tau)] = R_x(t+T, \tau).$$

Thus:

$$x(t) \text{ periodic} \implies x(t) \text{ WSC (cyclostationary)}.$$

The converse does not hold: cyclostationary signals need not be strictly periodic pointwise. For example, an amplitude-modulated process $x(t) = m(t) \cos(2\pi f_c t)$ with random data $m(t)$ can be cyclostationary with period $1/f_c$, while not being pointwise periodic.

3.2 Novel combined CSP, DSP approach to creating FPGA cache heirarchies

A function $f : A \rightarrow B$ is *foldable* iff $\exists A_0 \subseteq A, T = \{\tau_i : A \rightarrow A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

Where A_0 is called the *fundamental domain* & T , the set of transformations mapping from A_0 to A , the *recovery method*.

Motivating this definition with a classic example, consider $\sin(x)$ which has period 2π and can be represented with fundamental domain $\frac{\pi}{4}$ where the recovery method

$$T = \begin{cases} x, & 0 \leq x \leq \frac{\pi}{4}, \\ \pi - x, & \frac{\pi}{4} < x \leq \frac{\pi}{2}, \\ x - \pi, & \frac{\pi}{2} < x \leq \frac{3\pi}{2}, \\ 2\pi - x, & \frac{3\pi}{2} < x \leq 2\pi \end{cases}$$

$$f(x) = \begin{cases} f \circ \tau(x), & 0 \leq x \leq \pi, \\ -f \circ \tau(x), & \pi < x \leq 2\pi \end{cases}$$

Can be used to store effectively $4\times$ less memory M than the full period **OR** with the same memory footprint but $4\times$ greater granularity G . In general, the ratio between M & G , $\frac{M}{G}$, is called the *precision-memory gain tradeoff*. $M = \frac{P}{|A_0|}$, P is the period of the function.

Note: it is important that the recovery method $f \circ \tau(x)$ is relatively efficient / portable to an FPGA & that the tradeoff between the computational complexity of the recovery method & the memory gain M is not necessarily linear E.g. $\tau(x) \not\propto M$. A relevant example explored later on [**<empty citation>**] is the function $\arccos(x)$ which requires the computation of $\sqrt{x+1}$ on an FPGA in order to do the lookup on the reduced domain $\arccos : [0, \frac{1}{\sqrt{2}}] \rightarrow [-1, 1]$

3.2.1 Group-Theoretic Interpretation

The set of transformations T can be understood as the action of a *symmetry group* G on the domain A . In the sine example, the relevant group is the *dihedral group* D_4 , which consists of rotations and reflections of a square [**armstrong1988groups**,

gallian2021contemporary]. Each $\tau \in T$ corresponds to an element of D_4 acting on the domain of $\sin(x)$, with reflections accounting for sign changes and rotations corresponding to translations by $\pi/2$.

Thus, the concept of foldability is equivalent to requiring that the function f is invariant under the action of a group G , up to sign or phase factors. The *fundamental domain* A_0 is then a representative region of A under this group action, and folding reduces storage by exploiting orbit representatives of G .

In group-theoretic language, the prior example is invariant under the group action up to a sign factor, i.e.,

$$\sin(x) = \epsilon(\tau) \sin(\tau(x)), \quad \epsilon(\tau) \in \{+1, -1\}, \quad \tau \in G.$$

The fundamental domain A_0 corresponds to a set of orbit representatives of G acting on A , and the recovery method reconstructs the full signal from this reduced domain.

3.2.2 Link to Cyclostationary Signals

Foldable functions are naturally related to cyclostationarity. A cyclostationary signal $x(t)$ has statistics that are periodic with period T .

For a foldable function like $\sin(x)$, the fundamental domain A_0 can be interpreted as a segment over which the first- and second-order statistics repeat under the action of G . In other words, folding exploits the same periodic structure that cyclostationary analysis detects: the mean and autocorrelation within A_0 are sufficient to reconstruct the signal statistics over the entire domain.

Thus, foldability can be seen as a deterministic, structural analogue of cyclostationarity: instead of averaging over time to detect periodic statistics, we use group-theoretic transformations to reconstruct the full signal from a reduced representative domain.

Let $f : A \rightarrow B$ be a foldable function with fundamental domain $A_0 \subseteq A$ and recovery transformations $T = \{\tau_i : A \rightarrow A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

This implies that the function f is invariant under the group action of G generated by T , and the entire domain A can be reconstructed from A_0 using the transformations in T .

Now, consider the autocorrelation function of f :

$$R_f(x, \Delta) = \mathbb{E}[f(x)f^*(x + \Delta)].$$

Since $f(x) = f(\tau(x))$ for some $\tau \in T$, we have

$$R_f(x, \Delta) = \mathbb{E}[f(\tau(x))f^*(\tau(x + \Delta))].$$

Because $\tau(x), \tau(x + \Delta) \in A_0$ (up to another element of the group), all evaluations of $R_f(x, \Delta)$ are determined entirely by the behavior on A_0 . Therefore,

$$R_f(x, \Delta) = R_f(\tau(x), \Delta), \quad \forall x \in A, \tau \in T, \tau(x) \in A_0.$$

Thus, the fundamental domain A_0 captures all statistical information of the function f , analogous to how the fundamental domain in cyclostationary signal processing captures the periodic statistical properties of a signal.

3.2.3 Procedure for storing arbitrary signals

A procedure to *caching* an arbitrary signal $x(t)$ is the following:

1. Decompose the signal into a series of harmonics (for analysis purposes). In practice, this can be approximated via autocorrelation or spectral analysis rather than explicit FFT computation.
2. Use autocorrelation (ACF) analysis on the fundamental harmonic to estimate the period of a noisy signal within a specified SNR tolerance.
3. Iteratively refine the period estimate using heuristics, e.g., minimize spectral leakage if the period is undershot, or adjust lags using peak prominence methods.
4. Determine if the signal is *foldable* by checking whether there exists a fundamental domain A_0 and a set of transformations T such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \text{ and } f(x) = f \circ \tau(x).$$

This step is limited to signals with known symmetries (e.g., trigonometric functions, piecewise monotonic functions).

5. Construct a recovery method that maps any $x \in A$ to A_0 via the transformations in T , taking into account the desired precision-memory gain tradeoff based on control-theoretic or resource-demand considerations.
6. Implement the recovery method in a synthesizable and efficient form for FPGA deployment, ensuring that the computational complexity of applying $\tau(x)$ does not outweigh the memory savings.

Methods for (1), (2) and (3), previously discussed and implemented, are the subject of extensive optimization effort. Steps (4), (5), and (6), however, are considerably more difficult to achieve in practice on a completely unrestricted problem domain. Indeed, this paper focuses on a subset of signal definitions that can be reliably categorized, as well as on certain elementary signals that are relatively commonplace and often appear as components within more stochastic processes.

3.3 Synthesizable construction of the recovery method, T , for FPGA's

3.3.1 Efficient FPGA Implementation of $\arccos(x)$ Using 'Quarter'-Domain Folding

In previous sections, we discussed foldable functions and their use in memory-efficient lookup. In this section, we consider the case of $\arccos(x)$, which initially seems trivial but actually presents valuable insights for efficient computation on hardware.

Fundamental Domain Recovery The key identity used for recovery from a quarter-domain representation is:

$$\begin{aligned}\tau(x) &= \frac{1}{\sqrt{2}}\sqrt{x+1}, \\ f(x) &= 2 \cdot \arccos(\tau(x))\end{aligned}\tag{3.1}$$

$$\implies \arccos(x) = 2 \arccos\left(\frac{1}{\sqrt{2}}\sqrt{x+1}\right).\tag{3.2}$$

Applying a third-order Taylor series expansion to $\sqrt{x+1}$ yields:

$$16\sqrt{x+1} \approx x^3 - 2x^2 + 8x + 16.$$

Unlike the arcsin third-order polynomial, this polynomial can be factored over a finite field:

$$(x+1)(x^2+x+1).$$

To recover the original polynomial, we add the remaining series:

$$T_2 = -4x^2 + 7x + 15.$$

Efficient Hardware Computation Computing the first term:

$$T_1 = (x+1)(x^2+x+1)$$

(**Note** that the second term T_2 can be computed using precomputed values)

$$T_2 = -4x^2 + 7x + 15 = -4x^2 + 8x + 16 - (x+1) = -(x^2 \ll 2) + ((x+1)+1) \ll 3 - (x+1)$$

Which can be implemented as follows:

```
// Begin computation of tau(x) for arccos(x)
// arccos in [0, 1/sqrt(2)], Q2.21 format (24 bit data-width)

// Clock cycle 1: compute shared terms
t1 = x + 1;
x_squared = x * x;

// Clock cycle 2: compute T1
t1 = t1 * (x_squared + t1);

// Clock cycle 2: concurrent with T1, compute T2
t2 = -(x_squared << 2) + ((t1 + 1) << 3) - t1;

// Clock cycle 3: add finite-field-factorization correction term, T2, to T1
tau = t1 + t2;

// Clock cycle 4: right-shift by 4 to get sqrt(x + 1), multiply by 1/sqrt(2) term
localparam inv_sqrt2_q = ...; // 1/sqrt(2) stored in Q2.21 format
tau = (tau >> 4) * inv_sqrt2_q;
```

```
// Do the other required computations to get T

// Clock cycle 5/6: quantization of tau(x) to indice present in BRAM, LUT lookup
logic [23:0] T;
acos_tau_quantize f (.tau(tau), .f(T), .clk(clk));

// Clock cycle 7: final result
T = T << 1;
```

Resource Analysis and Benefits The total cost for this third-order implementation of $\tau(x)$:

- 3 multiplications (1 for shared term, 1 for T_1 , 1 for multiplication by $\frac{1}{\sqrt{2}}$)
- 6 additions/subtractions (1 for shared term, 1 for T_1 , 3 for T_2 , 1 for the final sum)
- 4 bit-shifts and 1 bitwise inversion (sign flip)

Advantages:

1. No transcendental operations in the argument apart from the $\frac{1}{\sqrt{2}}$ multiplier, simplifying quantization.
 - Quantization step can be bypassed to save a clock cycle
 - Achieved via computing the arccos lut values along a non-linear axis I.e. according to the distribution $\tau(x)$
2. Only a single DSP unit is required for all multiplies if the data width allows single-cycle multiplication.
3. No additional storage for constants required except for $\frac{1}{\sqrt{2}}$.
4. The computation is easily pipelined.
5. Bit-growth is predictable, scaling relative to $\deg(\tau(x))$

3.3.2 Generalizing recovery method construction technique

General procedure to obtain synthesizable FPGA code, given a known T

1. For some $\tau(x)$, express $\tau(x)$ with dyadic rational coefficients
2. Find a recursive finite factorization of $\mathbb{F}(\tau(x))$
3. Find and isolate shared/successive terms in $\mathbb{F}(\tau(x))$ [**reference needed**]
4. Pipeline and synthesize for target FPGA

3.3.3 Setup: Walsh basis and discrete (dyadic) synthesis

Let $\{W_k\}_{k \geq 0}$ denote the Walsh–Paley system on $[0, 1)$, an orthonormal basis of $L^2([0, 1))$ [Paley1932, Fine1949, GES1991]. For any $f \in L^2([0, 1))$, its Walsh expansion is

$$f(x) \sim \sum_{k=0}^{\infty} \widehat{f}(k) W_k(x), \quad \widehat{f}(k) := \int_0^1 f(t) W_k(t) dt.$$

Fix an integer $m \geq 1$ and set $N := 2^m$. Let the dyadic grid be

$$x_j := \frac{j}{N}, \quad j = 0, 1, \dots, N-1.$$

Define the (vector of) samples $y_j := f(x_j)$. The *discrete* Walsh–Hadamard synthesis uses the $N \times N$ Hadamard matrix H_N with entries $(H_N)_{k,j} = W_k(x_j) \in \{\pm 1\}$ (Paley ordering). Then the length- N coefficient vector $c = (c_k)_{k=0}^{N-1}$ is

$$c = \frac{1}{N} H_N y, \quad \text{i.e. } c_k = \frac{1}{N} \sum_{j=0}^{N-1} y_j W_k(x_j).$$

The length- N partial Walsh sum is the step function

$$S_N(x) := \sum_{k=0}^{N-1} c_k W_k(x),$$

which equals the conditional expectation of f onto the dyadic σ -algebra at scale 2^{-m} (the average of f on each dyadic subinterval).

3.3.4 Forcing dyadic coefficients via dyadic sampling & quantization

Step 1 (Dyadic quantizer). Fix $b \in \mathbb{N}$. Define the dyadic quantizer $Q_b : \mathbb{R} \rightarrow 2^{-b}\mathbb{Z}$ by

$$Q_b(u) := 2^{-b} \text{round}(2^b u).$$

Apply Q_b to the samples:

$$\tilde{y}_j := Q_b(f(x_j)) \in 2^{-b}\mathbb{Z}.$$

Step 2 (FWHT / discrete coefficients). Compute

$$\tilde{c}_k := \frac{1}{N} \sum_{j=0}^{N-1} \tilde{y}_j W_k(x_j).$$

Since each $\tilde{y}_j$ is a dyadic rational and $W_k(x_j) \in \{\pm 1\}$, we have

$$\tilde{c}_k \in 2^{-(b+m)}\mathbb{Z} \quad \Rightarrow \quad \tilde{c}_k \text{ is dyadic, } \forall k.$$

Thus the *quantized discrete Walsh expansion*

$$\tilde{S}_N(x) := \sum_{k=0}^{N-1} \tilde{c}_k W_k(x)$$

has *dyadic* coefficients and is piecewise constant on dyadic intervals.

3.3.5 Worked example: $f(x) = \sin(2\pi x)$

Take $f(x) = \sin(2\pi x)$, which has the non-dyadic Fourier series $f(x) = \frac{1}{2i}(e^{2\pi i x} - e^{-2\pi i x})$ (coefficients are not dyadic). Choose m, b and define

$$x_j = \frac{j}{2^m}, \quad \tilde{y}_j = Q_b(\sin(2\pi x_j)).$$

Compute Walsh coefficients

$$\tilde{c}_k = 2^{-m} \sum_{j=0}^{2^m-1} \tilde{y}_j W_k\left(\frac{j}{2^m}\right) \in 2^{-(b+m)}\mathbb{Z}.$$

Hence the Walsh approximation $\tilde{S}_{2^m}(x) = \sum_{k=0}^{2^m-1} \tilde{c}_k W_k(x)$ is a *dyadic series* that replaces the non-dyadic trigonometric series.

Error decomposition and guarantees

Write three errors:

$$\|f - \tilde{S}_N\|_{L^2} \leq \underbrace{\|f - \mathbb{E}[f | \mathcal{D}_m]\|_{L^2}}_{\text{dyadic averaging error}} + \underbrace{\|\mathbb{E}[f | \mathcal{D}_m] - \mathbb{E}[\tilde{f} | \mathcal{D}_m]\|_{L^2}}_{\text{sample quantization}} + \underbrace{\|\mathbb{E}[\tilde{f} | \mathcal{D}_m] - \tilde{S}_N\|_{L^2}}_{\text{discrete vs. continuous Walsh}},$$

where $\mathcal{D}_m$ is the dyadic σ -algebra at scale 2^{-m} and $\tilde{f}$ is the function matching the quantized samples on grid points and extended piecewise constantly on each dyadic interval.

- (*Averaging error*) $\mathbb{E}[f | \mathcal{D}_m]$ is the best L^2 -approximation of f by functions constant on dyadic intervals. As $m \rightarrow \infty$, $\|f - \mathbb{E}[f | \mathcal{D}_m]\|_{L^2} \rightarrow 0$ (martingale convergence; see [GES1991]). If f is Lipschitz with constant L , then

$$\|f - \mathbb{E}[f | \mathcal{D}_m]\|_{L^\infty} \leq L 2^{-m}.$$

- (*Quantization error*) Since $|\tilde{y}_j - y_j| \leq 2^{-b-1}$,

$$\|\mathbb{E}[f | \mathcal{D}_m] - \mathbb{E}[\tilde{f} | \mathcal{D}_m]\|_{L^2} \leq 2^{-b-1}.$$

- (*Discrete/continuous Walsh match*) For $N = 2^m$, the rectangle rule over dyadic intervals is *exact* for Walsh atoms W_k (they are constant on each interval), hence

$$\mathbb{E}[\tilde{f} | \mathcal{D}_m](x) = \sum_{k=0}^{N-1} \tilde{c}_k W_k(x) = \tilde{S}_N(x),$$

so this term vanishes.

Therefore, with $N = 2^m$,

$$\boxed{\|f - \tilde{S}_N\|_{L^2} \leq \|f - \mathbb{E}[f | \mathcal{D}_m]\|_{L^2} + 2^{-b-1}},$$

and every coefficient $\tilde{c}_k$ is dyadic: $\tilde{c}_k \in 2^{-(b+m)}\mathbb{Z}$.

Chapter 4

Architectural Design

The central challenge posed in this thesis is enabling *adaptive FPGA signal chains* that can predictively reconfigure hardware resources in response to changing workloads. A modular audio effect chain—comprising filters, nonlinear processors, and dynamic effects—places time-varying demands on FPGA primitives (DSP slices, LUTs, BRAMs). To meet this challenge, we require a mathematical framework that:

1. Quantifies the resource-error trade-off for individual DSP kernels (e.g., filters, oscillators).
2. Extends naturally to multi-rate and multi-band processing (wavelet decompositions).
3. Provides closed-form sensitivity metrics to guide runtime resource reallocation.
4. Enables dynamic bit allocation across frequency bands based on signal characteristics.

This chapter develops such a framework by unifying three key ideas: **(1)** projection-based approximation of infinite-support kernels (the sinc function), **(2)** sensitivity-driven bit allocation under hardware constraints, and **(3)** orthonormal wavelet bases that decompose signals into frequency bands with independent resource budgets. The result is an analytical tool that connects signal processing requirements directly to FPGA resource specifications, enabling the runtime optimization envisioned in Chapter 1.

Chapter roadmap. Section 4.1 establishes the optimal bit allocation problem for polynomial approximations of the sinc function, deriving a sensitivity matrix that guides resource distribution. Section 4.2 extends this to generic filter kernels via contour integration. Section 4.3 introduces dyadic quantization for fixed-point hardware. Section 4.4 extends the projector framework to the Shannon wavelet basis, enabling multi-band decomposition with per-scale bit allocation. Sections 4.5–4.6 synthesize these results into a hardware architecture for adaptive signal processing.

4.1 Optimal Coefficient Quantization via Projection Methods

One of the fundamental results in DSP is that the ideal low-pass filter (LPF) in the time domain is the sinc function [OppenheimDSP]:

$$h_{\text{ideal}}(t) = \text{sinc}(t) = \frac{\sin(t)}{t},$$

which corresponds to a perfect rectangular pulse in the frequency domain. However, $\text{sinc}(t)$ has infinite support with envelope decay $\propto 1/t$, making it impossible to implement directly in hardware. Practical implementations require both:

1. **Truncation:** approximating the infinite series by a finite polynomial of degree $2N$,
2. **Quantization:** representing each coefficient as a dyadic rational $a_n/2^{m_n}$ for fixed-point arithmetic.

This section establishes a unified framework for optimal bit allocation that minimizes the combined truncation and quantization error under hardware resource constraints.

4.1.1 Chapter Objectives

The goal of this chapter is to:

1. **Establish a projection-based error analysis framework** for the sinc function and its compositions/convolutions with arbitrary polynomials $p(z) \in \mathbb{C}[z]$.
2. **Derive an error projector** $R(z) : \mathbb{R} \rightarrow \mathbb{C}$ that simultaneously captures magnitude and phase error as functions of the truncation point N and bit allocation vector $\mathbf{m}$.
3. **Extend the projector to the Shannon wavelet orthonormal basis**, introducing a generalized frequency-domain kernel $\hat{H}(\omega)$ that enables sensitivity-based bit allocation across multiple frequency bands.
4. **Demonstrate computational savings** through concrete examples comparing the proposed method against naive uniform quantization and established filter design techniques.
5. **Validate against established methods** through quantitative metrics (SQNR, THD, resource utilization).

4.1.2 Scope and Limitations

This work focuses on bandlimited signals representable in the Shannon wavelet basis. The following are explicitly *outside* the scope:

- **Time-frequency trade-offs:** We do not address regularization between the Shannon basis (poor temporal, excellent frequency resolution) and bases like Haar functions (excellent temporal, poor frequency resolution).
- **Formal correctness proofs:** While the constructions are mathematically rigorous, we do not provide algorithmic correctness proofs to standards required for, e.g., verified compilation.
- **Parameter selection heuristics:** The choice of truncation point N and initial resolution j_0 is assumed given by application requirements (e.g., target SQNR, bandwidth).
- **Multi-band error correlation:** We treat each frequency band (wavelet scale) independently. Extensions to parametric EQ with inter-band correlation (e.g., modeling masking effects in psychoacoustics) are deferred to future work.

4.1.3 Problem Formulation

Consider a function $f(x)$ represented as an even polynomial series:

$$f(x) = \sum_{n=0}^N c_n x^{2n}, \quad (4.1)$$

where $c_n \in \mathbb{R}$ are the exact series coefficients (e.g., $c_n = \frac{(-1)^n}{(2n+1)!}$ for $\text{sinc}(x)$).

In fixed-point hardware implementations (FPGAs, ASICs), coefficients must be quantized to *dyadic rationals* [Gustafsson2007]:

$$\tilde{c}_n = \frac{a_n}{2^{m_n}}, \quad a_n \in \mathbb{Z}, \quad m_n \in \mathbb{N}_0, \quad (4.2)$$

where m_n is the number of fractional bits allocated to coefficient n , and $a_n = \lfloor c_n \cdot 2^{m_n} + 0.5 \rfloor$ is the nearest integer (rounding).

The coefficient-wise quantization error is:

$$\epsilon_n = c_n - \tilde{c}_n, \quad |\epsilon_n| \leq \frac{1}{2^{m_n+1}}. \quad (4.3)$$

The quantized polynomial is:

$$\tilde{f}(x) = \sum_{n=0}^N \tilde{c}_n x^{2n}, \quad (4.4)$$

and the total approximation error combines polynomial truncation and coefficient quantization:

$$E_{\text{total}}(x) = f(x) - \tilde{f}(x) = \sum_{n=0}^N \epsilon_n x^{2n}. \quad (4.5)$$

Optimization Problem

Objective: Determine the optimal bit allocation vector $\mathbf{m} = (m_0, m_1, \dots, m_N)$ that minimizes a weighted approximation error subject to a total bit budget:

$$\min_{\mathbf{m} \in \mathbb{N}_0^{N+1}} \mathcal{E}_{\text{proj}}(\mathbf{m}) \quad \text{subject to} \quad \sum_{n=0}^N m_n \leq B_{\text{total}}, \quad (4.6)$$

where $\mathcal{E}_{\text{proj}}(\mathbf{m})$ is an application-specific error metric (defined in Section 4.1.4).

Key insight: By formulating the problem in terms of a projection operator $R(x)$ derived from the target application (filter specifications, signal characteristics), we obtain a sensitivity matrix $\mathbf{S}$ that encodes both magnitude and phase error contributions. This enables:

- **Non-uniform bit allocation:** Coefficients with high sensitivity receive more bits.
- **Hardware efficiency:** Total bit budget reflects actual FPGA/ASIC resources (DSP slices, BRAM).
- **Extensibility:** The framework naturally extends to the Shannon wavelet basis, where each scale (frequency band) has an independent bit allocation sub-problem.

4.1.4 Projection Operator and Error Metric

Rather than minimizing approximation error in the standard L^2 norm (which treats all frequency components equally), we introduce a *projection operator* $R(x)$ that encodes application-specific error weighting. This formulation allows us to:

- Prioritize accuracy in critical frequency bands (e.g., passband for LPF design),
- Embed phase and magnitude sensitivities within a unified metric,
- Derive closed-form sensitivity expressions via contour integration (Section 4.1.9).

Projected Error Functional

The projected error is defined as the weighted inner product:

$$\mathcal{E}_{\text{proj}} = \langle E_{\text{total}}(x), R(x) \rangle = \int_0^{x_{\text{max}}} E_{\text{total}}(x) \cdot \overline{R(x)} dx, \quad (4.7)$$

where:

- $E_{\text{total}}(x)$ is the total approximation error from Eq. (4.5),
- $\overline{R(x)}$ denotes the complex conjugate of the projection operator,
- $\langle \cdot, \cdot \rangle$ represents the L^2 inner product on $[0, x_{\text{max}}]$,

- $x_{\max}$ is the maximum argument value of interest (e.g., $x_{\max} = \pi$ for audio applications).

Substituting the error polynomial from Eq. (4.5):

$$\mathcal{E}_{\text{proj}} = \int_0^{x_{\max}} \left[\sum_{n=0}^N \epsilon_n x^{2n} \right] \overline{R(x)} dx = \sum_{n=0}^N \epsilon_n \underbrace{\int_0^{x_{\max}} x^{2n} \overline{R(x)} dx}_{:= I_n}. \quad (4.8)$$

The integrals I_n quantify the contribution of coefficient n to the projected error and directly define the sensitivity matrix (Section 4.1.5).

4.1.5 Sensitivity Matrix Construction

The sensitivity of the projected error to coefficient n quantifies how a unit quantization error ϵ_n propagates through the system. From Eq. (4.8), we define:

$$S_n = \left| \frac{\partial \mathcal{E}_{\text{proj}}}{\partial \epsilon_n} \right| = \left| \int_0^{x_{\max}} x^{2n} \overline{R(x)} dx \right| = |I_n|. \quad (4.9)$$

The sensitivity S_n has a clear geometric meaning: it is the magnitude of the projection of the monomial x^{2n} onto the error weighting function $R(x)$ in the L^2 inner product space. Coefficients with larger sensitivity contribute more heavily to the overall error metric and therefore warrant higher bit precision.

Phase and Magnitude Decomposition

For a complex projector $R(x) = R_{\text{real}}(x) + iR_{\text{imag}}(x)$, the integral I_n decomposes into real and imaginary parts:

$$I_n^{\text{real}} = \int_0^{x_{\max}} x^{2n} \Re(R(x)) dx, \quad (4.10)$$

$$I_n^{\text{imag}} = \int_0^{x_{\max}} x^{2n} \Im(R(x)) dx. \quad (4.11)$$

Following the standard DSP convention, we associate:

$$S_n^{\text{mag}} = |I_n^{\text{real}}| \quad (\text{magnitude sensitivity}), \quad (4.12)$$

$$S_n^{\text{phase}} = |I_n^{\text{imag}}| \quad (\text{phase sensitivity}). \quad (4.13)$$

The complete sensitivity matrix $\mathbf{S} \in \mathbb{R}^{(N+1) \times 2}$ is:

$$\mathbf{S} = \begin{bmatrix} S_0^{\text{mag}} & S_0^{\text{phase}} \\ S_1^{\text{mag}} & S_1^{\text{phase}} \\ \vdots & \vdots \\ S_N^{\text{mag}} & S_N^{\text{phase}} \end{bmatrix}. \quad (4.14)$$

Application-Specific Weighting

The choice of projection operator $R(x)$ determines the error metric:

- **Uniform weighting:** $R(x) = 1$ yields the standard L^2 norm.
- **Frequency-selective weighting:** For LPF design, set $R(x) = \exp(i\omega_c x)$ to emphasize passband accuracy near cutoff frequency ω_c .
- **Wavelet-based weighting:** For multi-resolution analysis, $R(x)$ is the Shannon wavelet kernel at scale j (see Section ??).

4.1.6 Quantization Error Bounds

For dyadic quantization with m_n fractional bits allocated to coefficient n , the rounding operation in Eq. (4.2) satisfies:

$$|\epsilon_n| = |c_n - \tilde{c}_n| \leq \frac{1}{2^{m_n+1}}. \quad (4.15)$$

This bound is tight: the worst-case error occurs when c_n falls exactly midway between two quantization levels.

Worst-Case Projected Error

Applying the triangle inequality to Eq. (4.8):

$$|\mathcal{E}_{\text{proj}}| \leq \sum_{n=0}^N S_n |\epsilon_n| \leq \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}. \quad (4.16)$$

This upper bound is the objective function for bit allocation: by minimizing the right-hand side subject to $\sum_{n=0}^N m_n \leq B_{\text{total}}$, we obtain the allocation that guarantees the smallest worst-case projected error.

Expected Error (Uniform Random Rounding)

If quantization errors are modeled as independent uniform random variables $\epsilon_n \sim \mathcal{U}(-2^{-(m_n+1)}, 2^{-(m_n+1)})$, the expected squared error is:

$$\mathbb{E}[|\mathcal{E}_{\text{proj}}|^2] = \sum_{n=0}^N (S_n)^2 \cdot \frac{1}{3 \cdot 2^{2m_n+2}}, \quad (4.17)$$

where the factor $1/3$ arises from $\text{Var}[\epsilon_n] = \frac{1}{12} \cdot (2^{-m_n})^2$ [Widrow1996].

Remark: The greedy algorithm (Section 4.1.7) optimizes the worst-case bound (4.16), which is conservative but deterministic. For stochastic analysis, minimizing (4.17) leads to the allocation:

$$m_n^{\text{stochastic}} = m_{\text{base}} + \frac{1}{2} \log_2(S_n) + \text{const}. \quad (4.18)$$

(See Appendix ?? for detailed derivation.)

4.1.7 Optimal Bit Allocation Strategy

Given the sensitivity matrix $\mathbf{S}$ and total bit budget B_{total} , we now present three methods for determining the optimal bit allocation vector $\mathbf{m} = (m_0, m_1, \dots, m_N)$. Each method trades off computational complexity against optimality guarantees.

Method 1: Sensitivity-Proportional Allocation

The simplest heuristic allocates bits proportionally to sensitivity:

$$m_n = m_{\text{base}} + \left\lfloor \alpha \cdot \frac{S_n}{S_{\text{max}}} \right\rfloor, \quad (4.19)$$

where:

- $S_{\text{max}} = \max_{k \in \{0, \dots, N\}} S_k$ is the maximum sensitivity,
- m_{base} is the minimum bit allocation (typically $m_{\text{base}} = 8$ for 8-bit baseline precision),
- $\alpha \in [3, 8]$ controls the dynamic range of bit allocation (larger α increases disparity between high and low sensitivity coefficients).

After initial allocation, adjust α iteratively to satisfy the budget constraint:

$$\sum_{n=0}^N m_n = B_{\text{total}}. \quad (4.20)$$

Complexity: $O(N)$ per iteration; typically converges in 5–10 iterations via binary search on α .

Optimality: No guarantee; provides a reasonable starting point for refinement.

Method 2: Weighted Phase-Magnitude Allocation

For applications requiring different emphasis on phase versus magnitude accuracy (e.g., audio where phase distortion is perceptually critical), define a weighted sensitivity:

$$S_n^{\text{weighted}} = \sqrt{w_{\text{mag}}(S_n^{\text{mag}})^2 + w_{\text{phase}}(S_n^{\text{phase}})^2}, \quad (4.21)$$

where $w_{\text{mag}}, w_{\text{phase}} \geq 0$ are application-dependent weights satisfying $w_{\text{mag}} + w_{\text{phase}} = 1$.

Then apply proportional allocation using S_n^{weighted} in place of S_n in Eq. (4.19).

Typical values:

- Audio synthesis: $w_{\text{phase}} = 0.7, w_{\text{mag}} = 0.3$,
- Control systems: $w_{\text{mag}} = 0.8, w_{\text{phase}} = 0.2$,
- General filtering: $w_{\text{mag}} = w_{\text{phase}} = 0.5$.

Method 3: Greedy Optimization

For the provably optimal solution to minimize the worst-case bound (4.16), we employ a greedy algorithm [Gustafsson2007]. The algorithm proceeds as follows:

Algorithm 4.1: Greedy Bit Allocation

1. **Initialize:** Set $m_n = 1$ for all $n \in \{0, \dots, N\}$
2. **Set remaining budget:** $B_{\text{remaining}} = B_{\text{total}} - (N + 1)$
3. **While** $B_{\text{remaining}} > 0$:
 - (a) **Compute error reduction** for adding one bit to coefficient n :

$$\Delta_n = S_n \left(\frac{1}{2^{m_n+1}} - \frac{1}{2^{m_n+2}} \right) = \frac{S_n}{2^{m_n+2}}, \quad \forall n \in \{0, \dots, N\} \quad (4.22)$$

- (b) **Select coefficient** with maximum error reduction:

$$n^* = \arg \max_{n \in \{0, \dots, N\}} \Delta_n \quad (4.23)$$

- (c) **Increment allocation:** $m_{n^*} \leftarrow m_{n^*} + 1$
 - (d) **Decrement budget:** $B_{\text{remaining}} \leftarrow B_{\text{remaining}} - 1$
4. **Return** the optimal bit allocation vector $\mathbf{m} = (m_0, m_1, \dots, m_N)$

Complexity: $O(B_{\text{total}} \cdot N)$ — the outer loop executes $B_{\text{total}} - (N + 1)$ times, and each iteration scans $N + 1$ coefficients.

Optimality: The greedy algorithm is optimal for the additive error model in Eq. (4.16) because the objective function has the *diminishing returns property*: adding a bit to coefficient n reduces the error contribution by exactly half, independent of other allocations. (See Appendix ?? for formal proof.)

Method	Complexity	Optimal?	Use Case
Proportional (Sec. 4.1.7)	$O(N \log N)$	No	Rapid prototyping
Weighted (Sec. 4.1.7)	$O(N \log N)$	No	Phase-critical apps
Greedy (Sec. 4.1.7)	$O(B_{\text{total}} \cdot N)$	Yes	Production systems

Table 4.1: Comparison of bit allocation methods.

4.1.8 Evaluation of Projected Error

After determining the bit allocation vector $\mathbf{m}$ via one of the methods in Section 4.1.7, we compute the actual projected error to validate the design and compare against the theoretical bound (4.16).

Exact Projected Error

From Eq. (4.8), the projected error is:

$$\mathcal{E}_{\text{proj}} = \sum_{n=0}^N \epsilon_n \cdot I_n = \sum_{n=0}^N (c_n - \tilde{c}_n) \int_0^{x_{\max}} x^{2n} \overline{R(x)} dx. \quad (4.24)$$

where $\epsilon_n = c_n - \tilde{c}_n$ is the actual quantization error for coefficient n after rounding to m_n fractional bits.

Numerical Evaluation via Discretization

For practical computation, discretize the integral using a uniform grid with K points:

$$\mathcal{E}_{\text{proj}} \approx \sum_{n=0}^N \epsilon_n \sum_{k=0}^{K-1} x_k^{2n} \overline{R(x_k)} \Delta x, \quad (4.25)$$

where:

- $x_k = k \cdot \Delta x$ with $\Delta x = x_{\max}/K$,
- $K \geq 1024$ is typically sufficient for convergence (verify by doubling K and checking relative change $< 10^{-6}$).

Magnitude and Phase Error Components

Decompose the complex projected error into magnitude and phase contributions:

$$\mathcal{E}_{\text{real}} = \Re(\mathcal{E}_{\text{proj}}) = \sum_{n=0}^N \epsilon_n \cdot I_n^{\text{real}}, \quad (4.26)$$

$$\mathcal{E}_{\text{imag}} = \Im(\mathcal{E}_{\text{proj}}) = \sum_{n=0}^N \epsilon_n \cdot I_n^{\text{imag}}, \quad (4.27)$$

where I_n^{real} and I_n^{imag} are defined in Eqs. (4.10) and (4.11).

The magnitude error and phase error are then:

$$|\mathcal{E}_{\text{proj}}| = \sqrt{(\mathcal{E}_{\text{real}})^2 + (\mathcal{E}_{\text{imag}})^2}, \quad (4.28)$$

$$\angle \mathcal{E}_{\text{proj}} = \arctan\left(\frac{\mathcal{E}_{\text{imag}}}{\mathcal{E}_{\text{real}}}\right). \quad (4.29)$$

Validation Against Theoretical Bound

The computed error should satisfy the worst-case bound:

$$|\mathcal{E}_{\text{proj}}| \leq \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}, \quad (4.30)$$

with typical margin of 20%–50% (i.e., actual error is 0.5–0.8 times the theoretical bound). A larger margin indicates:

- **Conservative allocation:** The greedy algorithm over-allocates bits due to worst-case assumption; consider using the stochastic allocation (4.18) to reduce budget.
- **Favorable coefficient signs:** Cancellation between positive and negative ϵ_n terms reduces total error.

Practical Considerations for FPGA Implementation

When implementing the error evaluation in hardware:

1. **Precompute integrals I_n :** Store the $(N + 1)$ complex values in BRAM as a lookup table. Recomputation is unnecessary unless $R(x)$ or $x_{\max}$ changes.
2. **Use CORDIC for $R(x)$ evaluation:** If $R(x)$ involves trigonometric functions (e.g., $R(x) = e^{i\omega x}$), employ the CORDIC algorithm [Andraka1998, Volder1959] in rotation mode to avoid multiplier-intensive implementations.
3. **Pipeline the summation:** Implement Eq. (4.24) as a tree adder with $\lceil \log_2(N + 1) \rceil$ stages to achieve single-cycle throughput (latency $\propto \log_2 N$).
4. **Monitor at runtime:** For adaptive systems, recompute $\mathcal{E}_{\text{proj}}$ periodically (e.g., every 10^6 samples) and adjust $\mathbf{m}$ if error exceeds specification by triggering a reallocation cycle.

Connection to adaptive FPGA design. The bit allocation algorithms in this section directly address the resource management problem posed in the thesis abstract: given a time-varying audio signal with changing frequency content, the sensitivity matrix $\mathbf{S}$ provides real-time feedback on which polynomial coefficients require high precision. This enables the FPGA fabric to dynamically reallocate DSP slices and BRAM blocks—assigning more resources to high-sensitivity coefficients (critical frequency bands) while reducing precision for low-sensitivity terms. The greedy algorithm’s $O(B_{\text{total}} \cdot N)$ complexity makes it suitable for runtime execution, supporting the adaptive signal chain architecture described in Chapter 1.

However, the framework developed so far assumes that the projection operator $R(x)$ and sensitivity integrals $S_n = \left| \int_0^{x_{\max}} x^{2n} \overline{R(x)} dx \right|$ can be computed efficiently. Section 4.1.7 now derives an explicit closed-form expression for the sinc projector using complex analysis, which not only validates the abstract framework but also provides a computational template for evaluating any analytic kernel $H(\omega)$ via residue calculus (Section 4.2).

4.1.9 Projector Derivation for Sinc Function

The power series for $\text{sinc}(x)$ is given as:

$$\text{sinc}(x) = \frac{\sin(x)}{x} = \sum_{n=0}^{\infty} \frac{(-1)^n}{\Gamma(2n + 2)} x^{2n}.$$

Due to the necessity of considering truncated power series, we introduce the upper bound N . Re-indexing over $n \in \{0, \dots, 2N\}$ and using the Kronecker delta to select even terms:

$$\text{sinc}_N(x) = \sum_{n=0}^{2N} \delta_{n \equiv 0 \pmod{2}} \cdot \frac{(-1)^{n/2}}{\Gamma(n+2)} x^n = \sum_{n=0}^{2N} \delta_{n \equiv 0 \pmod{2}} \cdot \frac{i^n}{\Gamma(n+2)} x^n, \quad (4.31)$$

where the second equality holds because $i^n = (-1)^{n/2}$ when n is even.

Analytic Projector via Contour Integration

The introduction of the Kronecker delta $\delta_{n \equiv 0 \pmod{2}}$ serves dual purposes: selecting even coefficients and facilitating analytic continuation. To make this precise, we let $\Omega \subset \mathbb{C}$ be a simply connected domain containing $[0, x_{\max}]$. Consider the space:

$$\mathcal{A}(\Omega) = \{f : \Omega \rightarrow \mathbb{C} \mid f \text{ is holomorphic on } \Omega\}.$$

For $f \in \mathcal{A}(\Omega)$ with Taylor expansion about $z = 0$:

$$f(z) = \sum_{k=0}^{\infty} a_k z^k, \quad a_k = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(\zeta)}{\zeta^{k+1}} d\zeta,$$

where γ is a simple closed contour in Ω enclosing the origin.

The projector $\mathbb{P}_N : \mathcal{A}(\Omega) \rightarrow \mathcal{P}_{2N}^{\text{even}}$ is defined via the modified Cauchy integral:

$$\mathbb{P}_N[f](z) = \frac{1}{2\pi i} \oint_{\gamma} K_N(\zeta) f\left(\frac{z}{\zeta}\right) \frac{d\zeta}{\zeta}$$

where $K_N(\zeta)$ is the *selection kernel*:

$$K_N(\zeta) = \sum_{n=0}^N \zeta^{2n} = \frac{1 - \zeta^{2N+2}}{1 - \zeta^2}.$$

This kernel satisfies the projection property:

$$\frac{1}{2\pi i} \oint_{|\zeta|=1} K_N(\zeta) \zeta^{-k-1} d\zeta = \begin{cases} 1 & \text{if } k \in \{0, 2, 4, \dots, 2N\}, \\ 0 & \text{otherwise.} \end{cases}$$

Using the contour representation of the Kronecker delta (Lemma A.1.1), we can drop the Kronecker delta and express

$$\mathbb{P}_N[\text{sinc}](z\zeta) = \sum_{n=0}^{2N} \left[\frac{1}{2\pi i} \oint_{|\zeta|=r<1} \frac{1}{\zeta^{n+1}(1-\zeta^2)} d\zeta \right] \frac{i^n z^n}{\Gamma(n+2)}.$$

Since the sum is finite, we may interchange the sum and the integral:

$$\boxed{\mathbb{P}_N[\text{sinc}](z) = \frac{1}{2\pi i} \oint_{|\zeta|=r<1} \frac{1}{1-\zeta^2} \left[\sum_{n=0}^{2N} \frac{(iz/\zeta)^n}{\Gamma(n+2)} \right] \frac{d\zeta}{\zeta} \equiv \mathbb{P}_N[f](z)}$$

This contour-based projector inherently filters even coefficients—the only non-vanishing terms in the sinc expansion.

Evaluating the integral

$$\omega := \frac{iz}{\zeta}, \quad S_\infty(\omega) := \sum_{n=0}^{\infty} \frac{\omega^n}{\Gamma(n+2)}.$$

$$S_\infty(\omega) = \sum_{n=0}^{\infty} \frac{\omega^n}{(n+1)!} = \frac{1}{\omega} \sum_{k=1}^{\infty} \frac{\omega^k}{k!} = \frac{e^\omega - 1}{\omega},$$

with the removable value $S_\infty(0) = 1$.

Note: The swap of sum and integral is still justified since $|\zeta| = r > 0 \implies |\omega| = \frac{|z|}{r}$ is bounded, so the exponential series is uniformly convergent on the contour.

$$S_{2N}(\omega) := \sum_{n=0}^{2N} \frac{\omega^n}{\Gamma(n+2)} = \sum_{n=0}^{2N} \frac{\omega^n}{(n+1)!} = \frac{1}{\omega} \sum_{k=1}^{2N+1} \frac{\omega^k}{k!} = \frac{E_{2N+1}(\omega) - 1}{\omega},$$

where $E_m(z) := \sum_{k=0}^m \frac{z^k}{k!}$ is the truncated exponential polynomial.

$$\Gamma(m+1, z) = m! e^{-z} \sum_{k=0}^m \frac{z^k}{k!} \iff E_m(z) = \frac{e^z \Gamma(m+1, z)}{m!}.$$

Hence, with $m = 2N + 1$,

$$S_{2N}(\omega) = \frac{1}{\omega} \left(\frac{e^\omega \Gamma(2N+2, \omega)}{(2N+1)!} - 1 \right) = \frac{e^\omega \Gamma(2N+2, \omega)}{\omega (2N+1)!} - \frac{1}{\omega}.$$

$$S_\infty(\omega) - S_{2N}(\omega) = \frac{1}{\omega} \sum_{k=2N+2}^{\infty} \frac{\omega^k}{k!} \implies |S_\infty(\omega) - S_{2N}(\omega)| \leq \frac{e^{|\omega|}}{|\omega|} \frac{|\omega|^{2N+2}}{(2N+2)!}.$$

Performing a change of variable $\frac{iz}{\zeta} \rightarrow \omega$ on the integral & substituting:

$$\mathbb{P}_N[\text{sinc}](z) = \frac{1}{2\pi i} \oint_{|\omega|=|z|/r} \frac{1}{\omega^2 + z^2} \left[\frac{e^\omega \Gamma(2N+2, \omega)}{(2N+1)!} - 1 \right] d\omega$$

(??)

$$\mathbb{P}_N[\text{sinc}](z) = \frac{1}{2\pi i} \oint_{|\omega|=|z|/r} \frac{H(\omega)}{\omega^2 + z^2} d\omega, \quad 0 < r < 1,$$

with either choice

$$H(\omega) = E_{2N+1}(\omega) - 1 \quad \text{or} \quad H(\omega) = \frac{e^\omega \Gamma(2N+2, \omega)}{(2N+1)!} - 1,$$

and $E_{2N+1}(\omega) = \sum_{k=0}^{2N+1} \omega^k / k!$.

$$\omega^2 + z^2 = (\omega - iz)(\omega + iz), \quad \omega = \pm iz \text{ are simple poles inside the contour.}$$

For any analytic H ,

$$\text{Res}_{\omega=\pm iz} \frac{H(\omega)}{\omega^2 + z^2} = \frac{H(\pm iz)}{(\omega^2 + z^2)'} \Big|_{\omega=\pm iz} = \frac{H(\pm iz)}{2(\pm iz)}.$$

Note:

$$\lim_{N \rightarrow \infty} \mathbb{P}_N[\text{sinc}](z) = \frac{e^{iz} - e^{-iz}}{2iz} = \frac{\sin z}{z}.$$

This is as expected. The phasor error contribution is solely contingent upon the effect of truncation (I.e. spectral bleed/leakage)

Hence

$$\begin{aligned} \mathbb{P}_N[\text{sinc}](z) &= \frac{1}{2\pi i} \oint \frac{H(\omega)}{\omega^2 + z^2} d\omega = \frac{H(iz) - H(-iz)}{2iz} \\ &= \frac{E_{2N+1}(iz) - E_{2N+1}(-iz)}{2iz} = \sum_{m=0}^N \frac{(-1)^m z^{2m}}{(2m+1)!} \\ &= \frac{e^{iz}\Gamma(2N+2, iz) - e^{-iz}\Gamma(2N+2, -iz)}{2iz(2N+1)!}. \end{aligned}$$

$$\boxed{= \frac{\Im(e^{iz}\Gamma(2N+2, iz))}{z(2N+1)!}}.$$

This result is worth noting for the simplicity & expectedness of its decomposition. We showed, via. standard complex analytic methods, that the partial sum $\text{sinc}_N(z)$ can be rigorously expressed in it's incomplete upper gamma function form[**<empty citation>**]. Moreover, that the projector error itself $\mathbb{E}(\mathbb{P}_N)$, depends precisely on the upper gamma function's bound $N_{\text{gamma}} \propto N_{\text{sinc}}, N_{\text{gamma}} = 2(N_{\text{sinc}} + 1)$.

Lemma 4.1.1 (For $z \in \mathbb{R}$, the truncation projector can be written in terms of a hypergeometric series). **Important:** *This representation is valid only for $z \in \mathbb{R}$. For complex arguments $z \in \mathbb{C} \setminus \mathbb{R}$, the full form $\mathbb{P}_N[\text{sinc}](z) = \frac{\Im(e^{iz}\Gamma(2N+2, iz))}{z(2N+1)!}$ must be used to capture both magnitude and phase error. See Appendix A.1 for details.*

Proof. Let

$$\begin{aligned} \theta &:= iz, \quad z \in \mathbb{R} \\ S &:= 2(N+1) \end{aligned}$$

A holomorphic extension of the upper incomplete gamma function [NIST:DLMF] given as:

$$\begin{aligned} \gamma(S, \theta) &= \theta^S \Gamma(S) \gamma^*(S, \theta) \\ \Gamma(S, \theta) &= \Gamma(S) - \gamma(S, \theta) \\ \gamma^*(S, \theta) &:= e^{-\theta} \sum_{k=0}^{\infty} \frac{\theta^k}{\Gamma(S+k+1)} \end{aligned}$$

can be applied to the expression $\Im(e^{\theta}\Gamma(S, \theta))$ as follows:

$$\Im(e^\theta \Gamma(S, \theta)) = \Im(\Gamma(S)e^\theta) - \Im\left(\Gamma(S)\theta^S \sum_{k=0}^{\infty} \frac{\theta^k}{\Gamma(S+k+1)}\right)$$

For $z \in \mathbb{R}$, we have $\theta = iz \in i\mathbb{R}$, hence:

$$\Im(\Gamma(S)e^\theta) = \Gamma(S) \sin(z)$$

Since $S \in \mathbb{N}_0, S \geq 1 \implies \Gamma(S) = (S-1)!$ and $z \in \mathbb{R}$

$$\Gamma(S)\theta^S \sum_{k=0}^{\infty} \frac{\theta^k}{\Gamma(S+k+1)} = \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k \Gamma(S)}{\Gamma(S+k)} = \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k},$$

where $(S)_k := \frac{\Gamma(S+k)}{\Gamma(S)}$ is the rising Pochhammer symbol.

The outside term θ^{S-1} is imaginary-valued for $z \in \mathbb{R}$, since $\theta = iz$ and $S-1 = 2N+1$ is odd, hence $\theta^{S-1} = (iz)^{2N+1}$ is purely imaginary. We subsequently conclude, using the linearity of the $\Im$ operator over a series, that only even powers of θ contribute. Thus, for $z \in \mathbb{R}$:

$$\Im\left(\theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k}\right) = -i\theta^{S-1} \left(\sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k}\right) = -i\theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}}.$$

$\sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}}$ can be represented in terms of the general hypergeometric series $\mathcal{H}_S(z)$ [AndrewsAskey] as:

$$\mathcal{H}_S(z) := {}_1F_2\left(1; \frac{S}{2} + 1, \frac{S}{2} + \frac{3}{2}; -\frac{z^2}{4}\right).$$

$$\begin{aligned} \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}} &= -\frac{z^2}{S(S+1)} \mathcal{H}_S(z). \\ -i\theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}} &= \frac{i\theta^{S-1} z^2}{S(S+1)} \mathcal{H}_S(z) \equiv \frac{(-1)^{N+1} z^{S+1}}{S(S+1)} \mathcal{H}_S(z). \end{aligned}$$

$$\implies \Im(e^\theta \Gamma(2N+2, iz)) = \Gamma(S) \sin(z) - \frac{(-1)^{N+1} z^{S+1}}{S(S+1)} \mathcal{H}_S(z), \quad z \in \mathbb{R}$$

as desired. □

For which a table of values is given as reference (??)

Substituting values we finally arrive at the general solution for the projector onto the truncated power series of $\text{sinc}(x)$:

$$\begin{aligned} \mathbb{P}_N[\text{sinc}](z) &= \frac{1}{\Gamma(S)z} \left[\Gamma(S) \sin(z) - \frac{i\theta^{S-1} z^2}{S(S+1)} \mathcal{H}_S(z) \right] \\ &= \boxed{\text{sinc}(z) + \frac{(-1)^N z^S}{\Gamma(S+2)} \mathcal{H}_S(z)} \end{aligned}$$

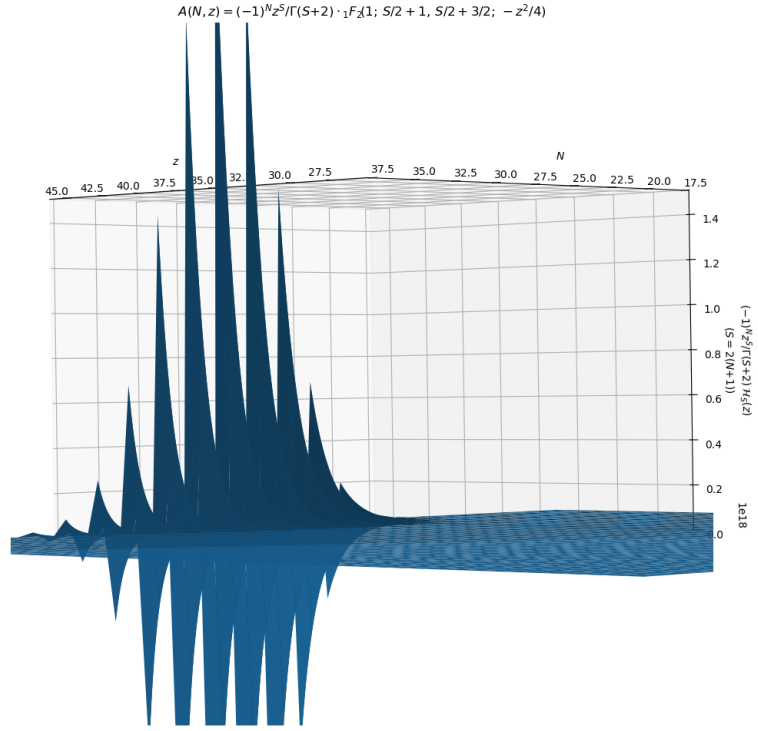


Figure 4.1: Plot of the remainder term $R_N(z)$

As expected:

$$R_N(z) := \text{sinc}(z) - \mathbb{P}_N[\text{sinc}](z) = \frac{(-1)^N z^S}{\Gamma(S+2)} \mathcal{H}_S(z)$$

4.2 Extending definition of the projector to accommodate generic $H(\omega)$

The projector derivation for $\text{sinc}(z)$ in Section 4.1.7 exploited a specific property: the sinc function’s Fourier transform is a rectangular pulse, which admits closed-form residue computation. However, audio signal chains require a much broader class of filter kernels—low-pass, high-pass, band-pass, parametric EQ, and nonlinear effects—each with distinct frequency responses $H(\omega)$.

The key insight is that *any analytic frequency response* $H(\omega)$ can be incorporated into the projection framework via contour integration, providing a unified computational method for evaluating sensitivity matrices across arbitrary filter types. This generalization is essential for the adaptive FPGA architecture: as the signal chain’s module graph evolves (e.g., user adds a resonant filter or switches from low-pass to band-pass), the system must rapidly recompute sensitivities to update bit allocations without redesigning the entire hardware pipeline.

This section establishes four design principles that motivate the generic kernel $H(\omega)$:

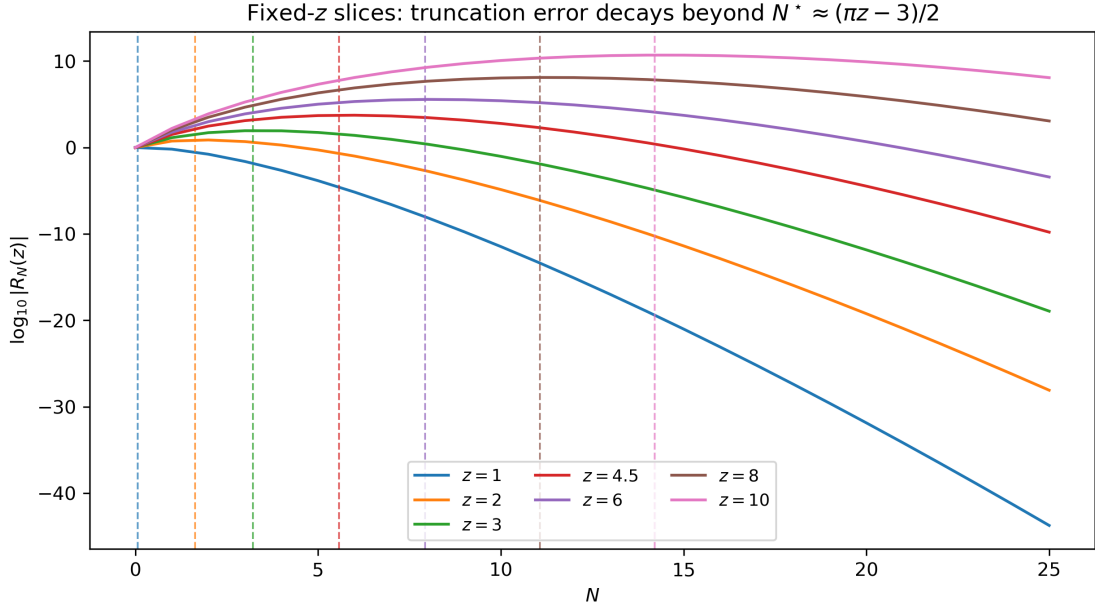


Figure 4.2: Logarithmic plot of the remainder term where the common definition given for $\text{sinc}, \text{sinc}(\pi z)$, is used. The decay point $N^* \approx \frac{\pi z - 3}{2} \propto z$ reflects the (*trivial*) fact that higher z needs a compensatory higher N for lower absolute error $|\mathbb{E}|$. **Note:** *Per(??) $\forall z \in a, |\mathbb{E}_N(z)| \gg \forall z \in b$ even though $|R_N(a)| < |R_N(b)|$ can hold true for $a \in [0, N^*), b \in [N^*, \infty)$*

1. **Residue-based evaluation:** Any analytic $H(\omega)$ can be evaluated systematically via its residues, enabling efficient hardware implementation through precomputed pole locations and CORDIC oscillators (Section 4.2.1).
2. **Dual error metrics:** The complex-valued projector encodes both magnitude and phase error, which are critical for audio applications where phase distortion is perceptually significant (Section 4.1.3).
3. **Composition for coefficient optimization:** Choosing $H(\omega)$ as $H(\omega) \circ p(z)$ (composition) minimizes per-term error in the power series itself, optimizing the polynomial approximation quality (useful for fixed-kernel designs).
4. **Convolution for application-specific weighting:** Choosing $H(\omega)$ as $H(\omega) \otimes p(z)$ (convolution) minimizes error relative to the target application's frequency response, enabling adaptive filtering where the kernel changes with signal characteristics (central to the adaptive signal chain goal).

The remainder of this section formalizes these principles and derives the general projector formula $\mathbb{P}_N[H](z)$ for arbitrary analytic kernels, laying the groundwork for the wavelet extension in Section 4.4.

4.2.1 Dyadic Quantization of Projector Coefficients

To implement the projector $\mathbb{P}_N[\text{sinc}](z) = \sum_{n=0}^N c_n z^{2n}$ in hardware, each coefficient $c_n = \frac{(-1)^n}{(2n+1)!}$ must be quantized to a dyadic rational [Gustafsson2007]. Let

$$\tilde{c}_n = \frac{a_n}{2^{m_n}}, \quad a_n \in \mathbb{Z}, \quad m_n \in \mathbb{N}_0$$

denote the quantized coefficient with m_n bits of fractional precision.

The *quantized projector* is then:

$$\tilde{\mathbb{P}}_N[\text{sinc}](z) = \sum_{n=0}^N \tilde{c}_n z^{2n} = \sum_{n=0}^N \frac{a_n}{2^{m_n}} z^{2n}. \quad (4.32)$$

The coefficient quantization error is $\epsilon_n = c_n - \tilde{c}_n$, satisfying

$$|\epsilon_n| \leq \frac{1}{2^{m_n+1}} \quad (4.33)$$

when a_n is chosen by rounding $c_n \cdot 2^{m_n}$ to the nearest integer [Widrow1996].

Total Error Decomposition

The complete approximation error combines *truncation error* (from projecting to degree $2N$) and *quantization error* (from dyadic rounding) [Kailath2000]:

$$\text{sinc}(z) - \tilde{\mathbb{P}}_N[\text{sinc}](z) = \underbrace{[\text{sinc}(z) - \mathbb{P}_N[\text{sinc}](z)]}_{\text{truncation: } R_N(z)} + \underbrace{[\mathbb{P}_N[\text{sinc}](z) - \tilde{\mathbb{P}}_N[\text{sinc}](z)]}_{\text{quantization: } Q_N(z)}. \quad (4.34)$$

From our earlier work, the truncation error is:

$$R_N(z) = \frac{(-1)^N z^{2(N+1)}}{\Gamma(2N+4)} \mathcal{H}_{2(N+1)}(z). \quad (4.35)$$

Remark. For $z \in \mathbb{R}$, the hypergeometric representation in Lemma 4.2.3.1 provides an alternative form that isolates magnitude error. For $z \in \mathbb{C} \setminus \mathbb{R}$, the full complex projector $\mathbb{P}_N[\text{sinc}](z) = \frac{\Im(e^{iz}\Gamma(2N+2, iz))}{z(2N+1)!}$ must be used to capture both magnitude and phase error. See Appendix A.1 for detailed derivations.

The quantization error is:

$$Q_N(z) = \sum_{n=0}^N \epsilon_n z^{2n}, \quad |Q_N(z)| \leq \sum_{n=0}^N \frac{|z|^{2n}}{2^{m_n+1}}. \quad (4.36)$$

Incorporating Quantization into the Kernel Framework

To unify truncation and quantization errors within the kernel framework, we introduce a *composite kernel* $\tilde{H}(\omega)$ that modulates the projector based on both truncation order N and bit allocation $\{m_n\}_{n=0}^N$.

Recall from Section 4.3 that the projector can be written in terms of a kernel $H(\omega)$ via residue calculus [Ahlfors1979]:

$$\mathbb{P}_N[\text{sinc}](z) = \frac{1}{2\pi i} \oint \frac{H(\omega)}{\omega^2 + z^2} d\omega$$

for appropriate contours and analytic H .

For the quantized projector, define the *quantization-weighted kernel*:

$$\tilde{H}_{\mathbf{m}}(\omega) = \sum_{n=0}^N \tilde{c}_n \frac{\omega^{2n}}{z^{2n}} \cdot H_n(\omega), \quad (4.37)$$

where $H_n(\omega)$ is a term-specific modulation kernel (e.g., frequency-dependent weighting) and $\mathbf{m} = (m_0, m_1, \dots, m_N)$ is the bit allocation vector.

The composite error kernel becomes:

$$\tilde{H}_{\mathbf{m}}(\omega) = H(\omega) - \sum_{n=0}^N \epsilon_n \cdot \Phi_n(\omega), \quad (4.38)$$

where $\Phi_n(\omega)$ is the frequency response contribution of the n -th term (detailed in Appendix A.2).

Sensitivity-Driven Bit Allocation with Quantization

Combining the sensitivity framework from Section 4.1.3 with dyadic quantization, the projected error becomes:

$$\mathcal{E}_{\text{total}} = \left| \int_0^{x_{\max}} [R_N(z) + Q_N(z)] \overline{R(z)} dz \right|, \quad (4.39)$$

where $R(z)$ is the application-specific projector from earlier.

By the triangle inequality:

$$\mathcal{E}_{\text{total}} \leq \underbrace{\left| \int_0^{x_{\max}} R_N(z) \overline{R(z)} dz \right|}_{\mathcal{E}_{\text{trunc}}} + \underbrace{\left| \int_0^{x_{\max}} Q_N(z) \overline{R(z)} dz \right|}_{\mathcal{E}_{\text{quant}}}. \quad (4.40)$$

The quantization error term expands as:

$$\begin{aligned} \mathcal{E}_{\text{quant}} &= \left| \sum_{n=0}^N \epsilon_n \int_0^{x_{\max}} z^{2n} \overline{R(z)} dz \right| \\ &\leq \sum_{n=0}^N |\epsilon_n| \cdot S_n = \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}, \end{aligned} \quad (4.41)$$

where $S_n = \left| \int_0^{x_{\max}} z^{2n} \overline{R(z)} dz \right|$ is the sensitivity from (4.9).

The optimal bit allocation minimizes $\mathcal{E}_{\text{quant}}$ subject to $\sum_{n=0}^N m_n \leq B_{\text{total}}$ by the greedy algorithm (Section 4.1.5.3) [VetterliKovacevic1995], now incorporating both truncation point N (which determines $\mathcal{E}_{\text{trunc}}$) and per-coefficient precision m_n (which determines $\mathcal{E}_{\text{quant}}$). The complete optimization procedure is detailed in Appendix A.3.

4.3 Shannon Wavelet Basis and Orthonormal Extension

Sections 4.1–4.3 established a projection framework for approximating individual filter kernels (sinc functions, generic $H(\omega)$) with optimal bit allocation under hardware constraints. However, real-world audio signals are *not* single-frequency tones—they are broadband signals with energy distributed across multiple octaves, where different frequency bands exhibit vastly different characteristics:

- **Bass frequencies (20–200 Hz):** High energy, perceptually critical, require high precision to avoid distortion.
- **Mid frequencies (200–2000 Hz):** Speech and most musical content; moderate precision adequate.
- **High frequencies (2–20 kHz):** Lower energy, often masked by louder sounds; can tolerate more quantization noise.

The adaptive FPGA signal chain envisioned in this thesis must allocate resources *per frequency band*, not uniformly across all frequencies. This requires decomposing the signal into localized frequency components—a task naturally suited to *wavelet bases*.

The Shannon wavelet basis provides an orthonormal decomposition of $L^2(\mathbb{R})$ where each wavelet $\psi_{j,k}(t)$ is localized to a specific octave band $[2^j\pi, 2^{j+1}\pi]$ in frequency. Critically, each band’s bit allocation problem *decouples*—we can independently optimize the bit vector $\mathbf{m}^{(j)}$ for scale j based on the signal’s energy in that band, enabling fine-grained resource control.

This section extends the projector framework $\mathbb{P}_N$ to the Shannon wavelet setting, deriving:

1. A *per-scale quantized projector* $\tilde{\psi}_{j,k}^{(N,\mathbf{m})}(t)$ with independent bit allocation $\mathbf{m}^{(j)}$ at each scale j ,
2. A *wavelet sensitivity matrix* $\mathbf{S}_{\text{wavelet}}^{(j)}$ that guides bit allocation within each frequency band,
3. A *hardware architecture* based on CORDIC oscillators and dyadic coefficients, suitable for FPGA implementation.

The result is a multi-resolution bit allocation scheme where the FPGA can dynamically adjust precision across frequency bands in response to signal changes—precisely the adaptive behavior required for the modular audio effect chain described in the abstract.

4.3.1 Shannon Wavelet Construction

The Shannon scaling function $\phi(t)$ and mother wavelet $\psi(t)$ are defined via their Fourier transforms:

$$\hat{\phi}(\omega) = \mathbf{1}_{[-\pi,\pi]}(\omega), \quad (4.42)$$

$$\hat{\psi}(\omega) = \mathbf{1}_{[-2\pi,-\pi] \cup [\pi,2\pi]}(\omega). \quad (4.43)$$

In the time domain, these correspond to:

$$\phi(t) = \text{sinc}_\pi(t) = \frac{\sin(\pi t)}{\pi t}, \quad (4.44)$$

$$\psi(t) = \frac{\sin(2\pi t) - \sin(\pi t)}{\pi t} = \text{sinc}_\pi(t/2) \cos(\pi t). \quad (4.45)$$

The family of dilated and translated wavelets forms an orthonormal basis for $L^2(\mathbb{R})$:

$$\psi_{j,k}(t) = 2^{j/2} \psi(2^j t - k), \quad j, k \in \mathbb{Z}, \quad (4.46)$$

together with the scaling functions at a fixed resolution j_0 :

$$\phi_{j_0,k}(t) = 2^{j_0/2} \phi(2^{j_0} t - k), \quad k \in \mathbb{Z}. \quad (4.47)$$

4.3.2 Orthonormality and Parseval Identity

The Shannon wavelet basis satisfies the orthonormality conditions [Daubechies1992]:

$$\langle \phi_{j_0,k}, \phi_{j_0,\ell} \rangle = \delta_{k,\ell}, \quad (4.48)$$

$$\langle \psi_{j,k}, \psi_{j',k'} \rangle = \delta_{j,j'} \delta_{k,k'}, \quad (4.49)$$

$$\langle \phi_{j_0,k}, \psi_{j,\ell} \rangle = 0 \quad \text{for all } j \geq j_0. \quad (4.50)$$

Any function $f \in L^2(\mathbb{R})$ can be decomposed as [Mallat1989]:

$$f(t) = \sum_{k \in \mathbb{Z}} c_{j_0,k} \phi_{j_0,k}(t) + \sum_{j=j_0}^{\infty} \sum_{k \in \mathbb{Z}} d_{j,k} \psi_{j,k}(t), \quad (4.51)$$

where the coefficients are given by:

$$c_{j_0,k} = \langle f, \phi_{j_0,k} \rangle = \int_{\mathbb{R}} f(t) \overline{\phi_{j_0,k}(t)} dt, \quad (4.52)$$

$$d_{j,k} = \langle f, \psi_{j,k} \rangle = \int_{\mathbb{R}} f(t) \overline{\psi_{j,k}(t)} dt. \quad (4.53)$$

The Parseval identity ensures energy preservation [Walter1992]:

$$\|f\|_{L^2}^2 = \sum_k |c_{j_0,k}|^2 + \sum_{j=j_0}^{\infty} \sum_k |d_{j,k}|^2. \quad (4.54)$$

4.3.3 Projector Extension to the Wavelet Basis

The key observation is that each basis element is a dilated and translated version of either $\text{sinc}_\pi(t)$ or products of sinc functions. Thus, the quantized projector $\tilde{\mathbb{P}}_N$ can be applied to each basis element independently.

Quantized Projected Scaling Functions

For the scaling function at resolution j_0 and position k , define:

$$\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) = 2^{j_0/2} \tilde{\mathbb{P}}_N[\text{sinc}_\pi](2^{j_0}t - k), \quad (4.55)$$

where $\tilde{\mathbb{P}}_N[\text{sinc}_\pi]$ is the quantized projector from (4.32) with N terms and bit allocation vector $\mathbf{m} = (m_0, \dots, m_N)$.

From the dyadic quantization framework, this becomes:

$$\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) = 2^{j_0/2} \sum_{n=0}^N \frac{a_n}{2^{m_n}} [\pi(2^{j_0}t - k)]^{2n}, \quad (4.56)$$

where $a_n = \left\lfloor \frac{(-1)^n \cdot 2^{m_n}}{(2n+1)!} \right\rfloor$ (nearest integer rounding).

Quantized Projected Wavelet Functions

For the mother wavelet $\psi(t) = \text{sinc}_\pi(t/2) \cos(\pi t)$, we use the product structure with independent quantization:

$$\tilde{\psi}_{j,k}^{(N,M,\mathbf{m},\mathbf{m}')} (t) = 2^{j/2} \tilde{\mathbb{P}}_N[\text{sinc}_\pi] \left(\frac{2^j t - k}{2} \right) \cdot \tilde{\mathbb{P}}_M[\cos](\pi(2^j t - k)), \quad (4.57)$$

where $\tilde{\mathbb{P}}_M[\cos]$ denotes the M -term quantized Taylor projection of the cosine with bit allocation $\mathbf{m}' = (m'_0, \dots, m'_M)$:

$$\tilde{\mathbb{P}}_M[\cos](x) = \sum_{m=0}^M \frac{b_m}{2^{m'_m}} x^{2m}, \quad b_m = \left\lfloor \frac{(-1)^m \cdot 2^{m'_m}}{(2m)!} \right\rfloor. \quad (4.58)$$

4.3.4 Approximation of Arbitrary Bandlimited Functions

Given a bandlimited function $f \in L^2(\mathbb{R})$ with Fourier support $\text{supp}(\hat{f}) \subset [-2^J\pi, 2^J\pi]$ for some finite J , the wavelet decomposition truncates naturally:

$$f(t) = \sum_{k=-K}^K c_{j_0,k} \phi_{j_0,k}(t) + \sum_{j=j_0}^J \sum_{k=-K_j}^{K_j} d_{j,k} \psi_{j,k}(t), \quad (4.59)$$

where K and K_j depend on the decay rate of the coefficients.

The quantized projected approximation is then:

$$\tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')} (t) = \sum_{k=-K}^K c_{j_0,k} \tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) + \sum_{j=j_0}^J \sum_{k=-K_j}^{K_j} d_{j,k} \tilde{\psi}_{j,k}^{(N,M,\mathbf{m},\mathbf{m}')} (t). \quad (4.60)$$

4.3.5 Error Analysis in the Wavelet Domain with Quantization

The total approximation error now combines truncation, quantization, and basis decomposition:

$$\|f - \tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')}\|_{L^2}^2 = \sum_k |c_{j_0,k}|^2 \|\phi_{j_0,k} - \tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}\|_{L^2}^2 + \sum_{j,k} |d_{j,k}|^2 \|\psi_{j,k} - \tilde{\psi}_{j,k}^{(N,M,\mathbf{m},\mathbf{m}')}\|_{L^2}^2. \quad (4.61)$$

For each basis element, the error decomposes as in (4.34). For the scaling function:

$$\begin{aligned} \|\phi_{j_0,k} - \tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}\|_{L^2}^2 &= 2^{j_0} \int_{\mathbb{R}} \left| R_N(\pi(2^{j_0}t - k)) + Q_N^{(\mathbf{m})}(\pi(2^{j_0}t - k)) \right|^2 dt \\ &\leq 2 \cdot 2^{j_0} \left[\int_{\mathbb{R}} |R_N(\pi u)|^2 du + \int_{\mathbb{R}} |Q_N^{(\mathbf{m})}(\pi u)|^2 du \right], \end{aligned} \quad (4.62)$$

where $R_N(z) = \frac{(-1)^N z^{2(N+1)}}{\Gamma(2N+4)} \mathcal{H}_{2(N+1)}(z)$ is the truncation remainder and $Q_N^{(\mathbf{m})}(z) = \sum_{n=0}^N \epsilon_n z^{2n}$ is the quantization error depending on bit allocation $\mathbf{m}$.

4.3.6 Sensitivity Matrix for the Wavelet Basis with Quantization

Extending the sensitivity framework to the wavelet domain with dyadic quantization, define the *wavelet sensitivity matrix* as:

$$\mathbf{S}_{\text{wavelet}} = \begin{bmatrix} \mathbf{S}_{\phi}^{(j_0)} & \mathbf{0} \\ \mathbf{0} & \text{block-diag}(\mathbf{S}_{\psi}^{(j)})_{j \geq j_0} \end{bmatrix}, \quad (4.63)$$

where each block $\mathbf{S}_{\phi}^{(j_0)}$ and $\mathbf{S}_{\psi}^{(j)}$ is itself a sensitivity matrix for the polynomial coefficients within that basis element.

For the scaling functions at resolution j_0 , the sensitivity of coefficient n is:

$$S_{\phi,n}^{(j_0)} = \max_k |c_{j_0,k}| \cdot 2^{j_0} \left| \int_{\mathbb{R}} u^{2n} \overline{R(\pi u)} du \right|, \quad (4.64)$$

where $R(z)$ is the application-specific projector from (4.7). This quantifies how quantization error ϵ_n in the n -th polynomial coefficient of $\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}$ affects the total projected error.

Similarly, for the wavelet functions at scale j :

$$S_{\psi,m}^{(j)} = \max_k |d_{j,k}| \cdot 2^j \left| \int_{\mathbb{R}} u^{2m} \cos(\pi \cdot 2^{j-j_0} u) \overline{R(\pi u)} du \right|. \quad (4.65)$$

The sensitivity-weighted bit allocation operates on the wavelet coefficient space via the composite error bound from (4.41), allocating precision based on:

1. The magnitude of the wavelet coefficients $|c_{j_0,k}|$ and $|d_{j,k}|$ (signal energy at each scale/position),

2. The projector order N and M at each scale (truncation error R_N, R_M),
3. The sensitivity $S_{\phi,n}^{(j_0)}$ and $S_{\psi,m}^{(j)}$ for each polynomial coefficient (quantization error weighting),
4. The total bit budget B_{total} distributed across all scales and positions.

The optimal bit allocation vector $\mathbf{m}^* = (\mathbf{m}_\phi, \{\mathbf{m}_\psi^{(j)}\}_{j \geq j_0})$ minimizes:

$$\sum_k |c_{j_0,k}| \sum_{n=0}^N \frac{S_{\phi,n}^{(j_0)}}{2^{m_{\phi,n}+1}} + \sum_{j,k} |d_{j,k}| \sum_{m=0}^M \frac{S_{\psi,m}^{(j)}}{2^{m_{\psi,m}^{(j)}+1}} \quad (4.66)$$

subject to the constraint:

$$\sum_{n=0}^N m_{\phi,n} + \sum_{j=j_0}^J \sum_{m=0}^M m_{\psi,m}^{(j)} \leq B_{\text{total}}. \quad (4.67)$$

4.3.7 Computational Advantages for Hardware Implementation

The Shannon wavelet decomposition offers several advantages for FPGA implementation [Andraka1998]:

Dyadic scaling. The dilation factor 2^j aligns naturally with binary arithmetic, allowing efficient bit-shifting operations rather than general multiplication [ParkhiUnbehauen1974].

Localized computation. Each wavelet $\psi_{j,k}$ has compact support in the frequency domain, meaning computations can be localized to specific frequency bands [Mallat1989].

This enables:

- *Parallel filter banks:* Different scales j can be processed in parallel [VetterliKovacevic1995].
- *Adaptive precision:* Higher frequencies (larger j) typically have smaller coefficients and can use reduced precision [Gustafsson2007].
- *Memory efficiency:* Only non-negligible coefficients need to be stored and processed [Burrus1998].

Fast wavelet transform. The Shannon wavelet admits a fast $O(N \log N)$ discrete wavelet transform via Mallat's algorithm [Mallat1989], which can be implemented using cascaded quadrature mirror filters (QMFs) [Vaidyanathan1993].

Direct frequency control. Because $\hat{\psi}_{j,k}(\omega) = 2^{-j/2} e^{-i\omega k/2^j} \mathbf{1}_{[-2^{j+1}\pi, -2^j\pi] \cup [2^j\pi, 2^{j+1}\pi]}(\omega)$, each wavelet scale j corresponds directly to an octave band $[2^j\pi, 2^{j+1}\pi]$ [Walter1992].

This enables:

- *Octave-band filtering:* Natural implementation of multi-rate filter banks [Crochiere1983].

- *Frequency-dependent bit allocation*: The bit allocation vector $\mathbf{m}$ can be chosen differently at each scale based on the frequency response requirements [JohnstonEtAl1997].
- *Adaptive FIR design*: The sensitivity matrix $\mathbf{S}_\psi^{(j)}$ guides coefficient quantization at each frequency band.

See Appendix A.4 for implementation details and hardware resource estimates.

4.3.8 Hardware-Efficient Kernel Selection with Quantization

For a given analytic kernel $H(\omega)$ in the frequency domain (as introduced in Section 4.3), combined with the quantization framework from Section 4.3.1, the wavelet-projected version becomes:

$$\tilde{f}_H^{(N,\mathbf{m})}(t) = \frac{1}{2\pi} \sum_{j=j_0}^J \sum_k d_{j,k} \int_{2^j\pi}^{2^{j+1}\pi} \tilde{H}_{\mathbf{m}}(\omega) \tilde{\Phi}_N^{(\mathbf{m})}((i\omega(2^{-j}t - k))^2) e^{i\omega t} d\omega, \quad (4.68)$$

where $\tilde{\Phi}_N^{(\mathbf{m})}$ is the quantized masked kernel incorporating dyadic coefficient rounding, and $\tilde{H}_{\mathbf{m}}(\omega)$ is the composite kernel from (4.38).

The kernel $H(\omega)$ can be chosen to optimize for [Parks1987, OppenheimSchafer2010]:

- *Passband ripple*: $H(\omega)$ constant on $[2^j\pi, 2^{j+1}\pi]$ for flat magnitude response.
- *Stopband attenuation*: $H(\omega)$ with sharp roll-off for selective frequency rejection [Rabiner1975].
- *Phase linearity*: $H(\omega) = |H(\omega)|e^{-i\omega\tau}$ for constant group delay τ [OppenheimSchafer2010].
- *Quantization robustness*: $H(\omega)$ designed to minimize sensitivity S_n at frequencies where quantization noise is most detrimental [JacksonEtAl1996].

The key result is that by expressing $\tilde{H}_{\mathbf{m}}(\omega)$ in terms of its residues (as established in Section 4.3) [Ahlfors1979], the hardware implementation reduces to:

$$\tilde{f}_H^{(N,\mathbf{m})}(t) = \sum_{j,k} d_{j,k} \sum_{\text{poles } \omega_p \in [2^j\pi, 2^{j+1}\pi]} \text{Res}_{\omega=\omega_p} \left[\frac{\tilde{H}_{\mathbf{m}}(\omega) \tilde{\Phi}_N^{(\mathbf{m})}((i\omega \cdot 2^{-j}t)^2) e^{i\omega t}}{(\omega - \omega_p)} \right] \quad (4.69)$$

which is a finite sum of complex exponentials with *dyadic rational coefficients*—exactly the form needed for efficient CORDIC-based oscillator implementations on FPGAs using fixed-point arithmetic [Volder1959, Andraka1998]. The bit allocation vector $\mathbf{m}$ determines the precision of each coefficient, allowing adaptive quantization based on the sensitivity matrix $\mathbf{S}_{\text{wavelet}}$.

The complete derivation of this residue-based decomposition is provided in Appendix A.5.

Synthesis of Sections 4.1–4.4. We have now developed all the analytical machinery needed for adaptive FPGA signal processing:

- **Section 4.1:** Bit allocation framework with sensitivity matrix $\mathbf{S}$ and greedy optimization.
- **Section 4.2:** Extension to arbitrary analytic kernels $H(\omega)$ via contour integration.
- **Section 4.3:** Dyadic quantization for fixed-point hardware, decomposing total error into truncation and quantization components.
- **Section 4.4:** Shannon wavelet basis enabling per-frequency-band bit allocation.

The remaining sections translate these theoretical results into a concrete hardware architecture. Section 4.5 describes how the quantized wavelet coefficients are embedded into a composite kernel for efficient FPGA implementation. Section 4.6 presents the complete dual bit-allocator/band-limiter system that adapts to signal characteristics in real time, fulfilling the adaptive signal chain vision outlined in the thesis abstract.

4.4 Composite Kernel Architecture for Wavelet-Based Filtering

[This section should describe the hardware architecture that implements the quantized Shannon wavelet projector, including:

- CORDIC oscillator banks for generating complex exponentials $e^{i\omega_p t}$
- Dyadic coefficient storage in BRAM with bit allocation vector $\mathbf{m}^{(j)}$ per scale
- Multi-rate processing: different sampling rates for different wavelet scales
- Pipeline architecture for tree-structured wavelet decomposition/reconstruction
- Resource budgeting: total DSP/BRAM/LUT allocation across all scales

]

4.5 Adaptive Bit Allocation and Band-Limiting for Real-Time Signal Processing

- Signal energy estimation per wavelet scale to detect frequency content changes
- Triggering conditions for bit reallocation (e.g., SQNR drops below threshold, energy shifts between bands)
- Fast reallocation algorithm: incremental updates to $\mathbf{m}$ without full recomputation

4.5. ADAPTIVE BIT ALLOCATION AND BAND-LIMITING FOR REAL-TIME SIGNAL PROC

- Band-limiting via dynamic truncation: disabling high-frequency scales when signal is low-pass
- Integration with modular effect chain: mapping DSP modules (reverb, distortion, EQ) to wavelet operations
- Case study: adaptive parametric EQ that adjusts bit allocation based on measured frequency response

The result is a complete FPGA architecture where:

1. Signal decomposition into Shannon wavelets provides natural frequency-band separation,
2. Per-band sensitivity matrices guide optimal bit allocation under total resource budget,
3. Runtime monitoring detects signal changes and triggers adaptive reallocation,
4. Hardware primitives (CORDIC, dyadic MAC) enable efficient implementation with predictable latency.

This realizes the thesis goal: an FPGA platform that *predictively reconfigures* in response to evolving audio workloads, supporting complex modular effect chains with runtime optimization.]

Chapter 5

Control theoretic model for dynamic FPGA resource balancing

Appendix A

Appendix

A.1 Auxiliary Lemmas

Lemma A.1.1 (Parity selector via contour integral). *For $n \in \mathbb{N}_0$ and any $0 < r < 1$, define*

$$I_n = \frac{1}{2\pi i} \oint_{|z|=r} \frac{dz}{z^{n+1}(1-z^2)}.$$

Then

$$I_n = \begin{cases} 1, & \text{if } n \text{ is even,} \\ 0, & \text{if } n \text{ is odd.} \end{cases}$$

Proof. Let $h(z) = \frac{1}{1-z^2}$. For $|z| < 1$,

$$h(z) = \sum_{m=0}^{\infty} z^{2m}.$$

Writing $h(z) = \sum_{k=0}^{\infty} a_k z^k$, we have $a_k = 1$ if k is even and $a_k = 0$ if k is odd. By Cauchy's coefficient formula,

$$I_n = \frac{1}{2\pi i} \oint_{|z|=r} \frac{h(z)}{z^{n+1}} dz = a_n,$$

so $I_n = 1$ when n is even and $I_n = 0$ when n is odd.

Residue viewpoint. The integrand $\frac{1}{z^{n+1}(1-z^2)}$ has singularities at $z = 0, \pm 1$, but for $0 < r < 1$ only $z = 0$ lies inside the contour. Hence the integral collapses to the single residue at $z = 0$:

$$I_n = \text{Res}_{z=0} \left(\frac{1}{z^{n+1}(1-z^2)} \right).$$

Expanding $\frac{1}{1-z^2} = \sum_{m=0}^{\infty} z^{2m}$ gives the Laurent series

$$\frac{1}{z^{n+1}(1-z^2)} = \sum_{m=0}^{\infty} z^{2m-n-1},$$

so the residue (coefficient of z^{-1}) is 1 iff $2m = n$, and 0 otherwise. Equivalently, using $\text{Res}_{z=0} \left(\frac{h(z)}{z^{n+1}} \right) = \frac{h^{(n)}(0)}{n!}$ for analytic h with $h(z) = \frac{1}{1-z^2}$, we have $h^{(n)}(0) = n!$ for even n and 0 for odd n , yielding the same conclusion. $\square$

Lemma A.1.2 ($\mathcal{H}_S(z)$ does not allow any further elementary decomposition). *Proof.* It is known that any ${}_3F_2$ series with the same argument, with parameters differing by integers, are linearly related[**<empty citation>**]

Let $x := -\frac{z^2}{4}$.

$$x \mathcal{H}_{S+2}(z) = \left(\frac{S}{2} + 1 \right) \left(\frac{S}{2} + \frac{3}{2} \right) [\mathcal{H}_S(z) - 1]. \quad (\text{R+})$$

$$x \mathcal{H}_S(z) = \left(\frac{S}{2} \right) \left(\frac{S}{2} + \frac{1}{2} \right) [\mathcal{H}_{S-2}(z) - 1]. \quad (\text{R-})$$

$$\left(\frac{S}{2} + 1 \right) \left(\frac{S}{2} + \frac{3}{2} \right) \mathcal{H}_{S+2}(z) - \left(\frac{S}{2} \right) \left(\frac{S}{2} + \frac{1}{2} \right) \mathcal{H}_{S-2}(z) = - \left[\left(\frac{S}{2} \right) \left(\frac{S}{2} + \frac{1}{2} \right) + x \right]. \quad (\text{C})$$

$$\left[\phi(\phi + b - 1)(\phi + c - 1) - x(\phi + 1) \right] y = 0, \quad \phi := x \frac{d}{dx}, \quad b = \frac{S}{2} + 1, \quad c = \frac{S}{2} + \frac{3}{2}.$$

$\square$

A.2 Hypergeometric Form for Real Arguments

Restriction to Real Domain. The hypergeometric representation of the truncation remainder derived in Lemma 4.2.3.1:

$$\mathbb{P}_N[\text{sinc}](z) = \text{sinc}(z) + \frac{(-1)^N z^{2(N+1)}}{\Gamma(2N+4)} \mathcal{H}_{2(N+1)}(z)$$

with

$$\mathcal{H}_S(z) = {}_1F_2 \left(1; \frac{S}{2} + 1, \frac{S}{2} + \frac{3}{2}; -\frac{z^2}{4} \right)$$

is valid *only* for $z \in \mathbb{R}$.

Proof of restriction. In the derivation, we used:

$$\Im \left(\theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k} \right) = -i \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}}, \quad \theta = iz, \quad S = 2(N+1).$$

This step relies on the fact that $\theta^{S-1} = (iz)^{2N+1}$ is purely imaginary (since $2N+1$ is odd), allowing us to conclude that only even powers of θ contribute to $\Im(\cdot)$.

However, for $z \in \mathbb{C} \setminus \mathbb{R}$, we have $\theta = iz \notin i\mathbb{R}$, so θ^{2k} is *not* purely real, and the separation:

$$\Im\left(\sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k}\right) = (\text{only even } k \text{ terms})$$

fails. The correct approach for complex z is to retain the full complex form:

$$\mathbb{P}_N[\text{sinc}](z) = \frac{\Im(e^{iz}\Gamma(2N+2, iz))}{z(2N+1)!}, \quad z \in \mathbb{C}.$$

A.3 Shannon Wavelet Derivations

Phase error for complex z . For $z = x + iy$ with $y \neq 0$:

$$e^{iz} = e^{i(x+iy)} = e^{ix-y},$$

$$\Gamma(2N+2, iz) = \Gamma(2N+2) - \gamma(2N+2, iz),$$

where $\gamma(s, w) = \int_0^w t^{s-1}e^{-t} dt$ is the lower incomplete gamma function. The imaginary part captures both:

- *Magnitude error*: from truncating the Taylor series to N terms.
- *Phase error*: from the complex exponential e^{iz} interacting with the incomplete gamma function.

Thus, for applications requiring phase accuracy (e.g., complex filters, IQ modulation), the full complex projector must be used, and the hypergeometric form is *only* a computational simplification for real-valued signals.

A.3.1 Frequency Response Contribution $\Phi_n(\omega)$

The term-specific frequency contribution $\Phi_n(\omega)$ in (4.38) arises from the Fourier transform of the n -th basis monomial x^{2n} .

For a polynomial projector:

$$\tilde{\mathbb{P}}_N[f](x) = \sum_{n=0}^N \tilde{c}_n x^{2n},$$

the frequency response is:

$$\widehat{\tilde{\mathbb{P}}_N[f]}(\omega) = \sum_{n=0}^N \tilde{c}_n \cdot \mathcal{F}\{x^{2n}\}(\omega),$$

where $\mathcal{F}\{x^{2n}\}(\omega) = i^{2n} \cdot 2\pi\delta^{(2n)}(\omega)$ is the $(2n)$ -th distributional derivative of the Dirac delta.

For practical filter design, we regularize by restricting to a compact domain $[0, x_{\max}]$:

$$\Phi_n(\omega) = \int_0^{x_{\max}} x^{2n} e^{-i\omega x} dx = \frac{\partial^{2n}}{\partial(i\omega)^{2n}} \left[\frac{e^{-i\omega x_{\max}} - 1}{-i\omega} \right].$$

Using integration by parts $2n$ times:

$$\Phi_n(\omega) = \sum_{k=0}^{2n} \binom{2n}{k} \frac{(2n-k)!}{\omega^{2n-k+1}} \left[(-i)^k e^{-i\omega x_{\max}} x_{\max}^k - (-i)^k \delta_{k,0} \right],$$

which exhibits $O(\omega^{-(2n+1)})$ decay, ensuring well-defined frequency localization.

A.3.2 Greedy Bit Allocation with Truncation-Quantization Trade-off

The complete optimization problem is:

$$\min_{\substack{N \in \mathbb{N}_0 \\ \mathbf{m} \in \mathbb{N}_0^{N+1}}} [\mathcal{E}_{\text{trunc}}(N) + \mathcal{E}_{\text{quant}}(N, \mathbf{m})] \quad \text{s.t.} \quad \sum_{n=0}^N m_n \leq B_{\text{total}}. \quad (\text{P})$$

where:

$$\mathcal{E}_{\text{trunc}}(N) = \left| \int_0^{x_{\max}} R_N(z) \overline{R(z)} dz \right|,$$

$$\mathcal{E}_{\text{quant}}(N, \mathbf{m}) = \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}.$$

Algorithm.

1. **Outer loop (choose N):** For $N = 0, 1, 2, \dots, N_{\max}$:

- (a) Compute truncation error $\mathcal{E}_{\text{trunc}}(N)$.
- (b) Compute sensitivities S_n for $n = 0, \dots, N$.
- (c) Solve the inner bit allocation problem for this N .

2. **Inner loop (greedy bit allocation for fixed N):**

- (a) Initialize $m_n = 1$ for all $n \in \{0, \dots, N\}$.
- (b) Set $B_{\text{remaining}} = B_{\text{total}} - (N + 1)$.
- (c) While $B_{\text{remaining}} > 0$:
 - Compute marginal error reduction:

$$\Delta_n = S_n \left(\frac{1}{2^{m_n+1}} - \frac{1}{2^{m_n+2}} \right) = \frac{S_n}{2^{m_n+2}}.$$
 - Find $n^* = \arg \max_n \Delta_n$.
 - Update: $m_{n^*} \leftarrow m_{n^*} + 1$, $B_{\text{remaining}} \leftarrow B_{\text{remaining}} - 1$.
- (d) Return $\mathcal{E}_{\text{quant}}(N, \mathbf{m})$.

3. **Select optimal N^* :**

$$N^* = \arg \min_N [\mathcal{E}_{\text{trunc}}(N) + \mathcal{E}_{\text{quant}}(N, \mathbf{m}^*(N))].$$

Complexity. The outer loop runs $O(N_{\max})$ times. For each N , the inner greedy algorithm runs $O(B_{\text{total}} \cdot N)$ iterations. Each iteration requires $O(N)$ comparisons to find $\arg \max$. Total: $O(N_{\max} \cdot B_{\text{total}} \cdot N^2)$, which can be optimized to $O(N_{\max} \cdot B_{\text{total}} \cdot N \log N)$ using a priority queue.

A.3.3 Hardware Resource Estimates for Shannon Wavelet Implementation

FPGA resource model. Consider a Xilinx Virtex-7 FPGA with the following primitives:

- DSP48E1 slices: 18×25 -bit multipliers, 48-bit accumulators.
- Block RAM (BRAM): 36 Kb dual-port memory blocks.
- Look-up tables (LUTs): 6-input logic functions.

Scaling function computation. For $\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t)$:

$$\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) = 2^{j_0/2} \sum_{n=0}^N \frac{a_n}{2^{m_n}} [\pi(2^{j_0}t - k)]^{2n}.$$

Resources per basis function:

- $(N+1)$ coefficient storage: $(N+1) \times \max_n(m_n + \lceil \log_2 |a_n| \rceil)$ bits in BRAM.
- Power computation u^{2n} for $u = \pi(2^{j_0}t - k)$: $\lceil \log_2(N+1) \rceil$ DSP slices (using Horner's method or parallel squaring tree).
- Multiply-accumulate: $(N+1)$ DSP slices or time-multiplexed over $\lceil (N+1)/P \rceil$ cycles for P parallel DSPs.
- Dyadic scaling $\frac{a_n}{2^{m_n}}$: implemented as barrel shifter using $O(m_n)$ LUTs (no DSP needed).

Example ($N=4$, $m_n \in [8, 16]$):

- Coefficients: 5×24 bits = 120 bits ≈ 0.003 BRAM.
- Power computation: 3 DSP slices (parallel).
- MAC: 5 DSP slices (fully parallel) or 1 DSP slice (5-cycle latency).
- Total per basis: 8 DSP, 120-bit BRAM, ~ 50 LUTs.

For a 3-scale wavelet decomposition (j_0, j_0+1, j_0+2) with $K = 32$ spatial positions per scale:

- Total DSP: $8 \times 32 \times 3 = 768$ DSPs (parallel) or $8 \times 3 = 24$ DSPs (time-multiplexed over 32 cycles).

- Total BRAM: $120 \times 32 \times 3 = 11520$ bits ≈ 0.32 BRAM blocks.
- Throughput: 1 sample per cycle (parallel) or 1 sample per 32 cycles (multiplexed).

Virtex-7 XC7VX485T has 2800 DSP slices and 1030 BRAM blocks, so even the fully parallel implementation uses $\sim 27\%$ DSP and $< 1\%$ BRAM.

A.3.4 Residue-Based Decomposition for Hardware Kernels

Consider a rational kernel:

$$H(\omega) = \frac{P(\omega)}{Q(\omega)}, \quad Q(\omega) = \prod_{p=1}^M (\omega - \omega_p)^{r_p},$$

where $\omega_p \in \mathbb{C}$ are the poles with multiplicities r_p .

The wavelet-projected integral:

$$I_j(t) = \int_{2^j\pi}^{2^{j+1}\pi} H(\omega) \tilde{\Phi}_N^{(\mathbf{m})}((i\omega \cdot 2^{-j}t)^2) e^{i\omega t} d\omega$$

can be evaluated by:

1. Extend the integration domain to a contour Γ_j enclosing $[2^j\pi, 2^{j+1}\pi]$ and all poles ω_p in this range.
2. Apply the residue theorem:

$$I_j(t) = 2\pi i \sum_{\omega_p \in [2^j\pi, 2^{j+1}\pi]} \text{Res}_{\omega=\omega_p} \left[H(\omega) \tilde{\Phi}_N^{(\mathbf{m})}((i\omega \cdot 2^{-j}t)^2) e^{i\omega t} \right].$$

t

For a simple pole ω_p (i.e., $r_p = 1$):

$$\text{Res}_{\omega=\omega_p} \left[\frac{P(\omega)}{Q(\omega)} F(\omega) \right] = \frac{P(\omega_p)}{Q'(\omega_p)} F(\omega_p),$$

where $F(\omega) = \tilde{\Phi}_N^{(\mathbf{m})}((i\omega \cdot 2^{-j}t)^2) e^{i\omega t}$.

Expanding $\tilde{\Phi}_N^{(\mathbf{m})}(w) = \sum_{n=0}^N \frac{a_n}{2^{m_n}} \frac{w^n}{(2n)!}$:

$$\begin{aligned} F(\omega_p) &= \sum_{n=0}^N \frac{a_n}{2^{m_n}(2n)!} [(i\omega_p \cdot 2^{-j}t)^2]^n e^{i\omega_p t} \\ &= e^{i\omega_p t} \sum_{n=0}^N \frac{a_n(-1)^n}{2^{m_n}(2n)!} \left(\frac{\omega_p t}{2^j} \right)^{2n}. \end{aligned}$$

Thus:

$$I_j(t) = 2\pi i \sum_{\omega_p} \underbrace{\frac{P(\omega_p)}{Q'(\omega_p)}}_{\text{constant}} \cdot \underbrace{\sum_{n=0}^N \frac{a_n(-1)^n}{2^{m_n}(2n)!} \left(\frac{\omega_p t}{2^j} \right)^{2n}}_{\text{dyadic polynomial in } t}.$$

t

Hardware implementation.

- Precompute constants $C_p = 2\pi i \frac{P(\omega_p)}{Q'(\omega_p)}$ (stored in BRAM).
- Oscillator: Generate $e^{i\omega_p t}$ using CORDIC in rotation mode (1 DSP per pole).
- Polynomial: Evaluate the dyadic polynomial using Horner's method (N DSPs or 1 DSP with N -cycle latency).
- Accumulate: Sum over all poles p (1 DSP).

For M poles and N terms: $M(N + 1) + 1$ DSP slices (parallel) or $N + 2$ DSP slices (time-multiplexed over M poles).

This architecture naturally supports frequency-dependent bit allocation: high-sensitivity poles can use larger m_n for their polynomial coefficients, while low-sensitivity poles use reduced precision, all determined by the sensitivity matrix $\mathbf{S}_{\text{wavelet}}$.

A.3.5 Signal Synthesis Capabilities of Shannon Wavelet Decomposition

t

What Can Be Synthesized

The Shannon wavelet framework with n scales starting at resolution j_0 can exactly represent any bandlimited signal with Fourier support in $[-2^{j_0+n}\pi, 2^{j_0+n}\pi]$. This is the Shannon-Nyquist sampling theorem extended to wavelet bases [Daubechies1992].

Fundamental Limitation: Bandlimited Signals Only. Unlike time-domain polynomial approximation or windowed sinc functions, the Shannon wavelet *cannot* represent:

- Sharp transients (step functions, clicks, impulses)
- Signals with infinite bandwidth
- Non-smooth waveforms (square waves, sawtooth without filtering)

However, *any practical audio or RF signal* is effectively bandlimited by:

- Physical limits (speaker/microphone bandwidth, antenna bandwidth)
- Anti-aliasing filters before sampling
- Nyquist limit of the sampling rate

Concrete Synthesis Examples

Example 1: Additive synthesis (organ, FM synthesis). Synthesize a tone with 8 harmonics at frequency f_0 :

$$s(t) = \sum_{k=1}^8 A_k \sin(2\pi k f_0 t + \phi_k).$$

For $f_0 = 440$ Hz (A4) with 8 harmonics, highest frequency is $8 \times 440 = 3520$ Hz.
Shannon wavelet representation:

- Choose j_0 such that $2^{j_0}\pi/T_s \approx 220$ Hz (below fundamental).
- 3-scale decomposition covers up to $8 \times 2^{j_0}\pi/T_s \approx 7$ kHz.
- Each harmonic k lives in scale $j = j_0 + \lfloor \log_2(k) \rfloor$.
- Total coefficients: ~ 16 per harmonic $\times 8$ harmonics = 128 coefficients.

Computational savings vs. time-domain:

- Time-domain sinc interpolation: requires $O(L)$ samples over support $L \approx$ several periods.
- Shannon wavelet: $O(8)$ wavelets (one per harmonic) with compact frequency support.
- **Advantage:** Adding/removing harmonics is trivial (set coefficient to zero), enabling real-time additive synthesis with frequency-specific envelope control.

Example 2: Subtractive synthesis (sawtooth $\rightarrow$ lowpass filter). Start with a bandlimited sawtooth (infinite harmonics, amplitude $\propto 1/k$):

$$\text{sawtooth}(t) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} \sin(2\pi k f_0 t).$$

Bandlimit to 32 harmonics (typical for audio):

- 5-scale Shannon decomposition ($j_0 = 0$ to $j_0 + 4$) covers 32 octaves.
- Apply a 2-pole lowpass filter $H(\omega) = \frac{1}{(1+i\omega/\omega_c)^2}$ with cutoff ω_c .
- In the wavelet domain, this is a *diagonal scaling* of coefficients:

$$d_{j,k}^{\text{filtered}} = H(2^{j+1/2}\pi) \cdot d_{j,k}^{\text{original}}.$$

- **Result:** Filter is a single multiplication per scale, not a convolution.

Hardware advantage:

- Time-domain FIR: $O(N_{\text{taps}})$ MACs per sample (e.g., 128-tap filter = 128 MACs).
- Shannon wavelet: 5 complex multiplications (one per scale), then reconstruct.
- For real-time filter sweeps (cutoff $\omega_c(t)$ varies), only the scale weights change—no filter redesign needed.

Example 3: Phase vocoder / time-stretching. Time-stretch a signal by factor α without changing pitch:

- Decompose into Shannon wavelets $\{d_{j,k}\}$.
- Modify temporal positions: $\psi_{j,k}(t) \rightarrow \psi_{j,k}(t/\alpha - k)$ (dilation in time).
- Frequency content (pitch) is unchanged because wavelets are shift-invariant in frequency.

Traditional phase vocoder:

- STFT (Short-Time Fourier Transform): window size trades off time vs. frequency resolution.
- Shannon wavelets: *native* multi-resolution—low frequencies have long windows, high frequencies have short windows automatically.

Example 4: Parametric EQ (audio mixing). Boost/cut at specific frequency bands (e.g., bass boost at 100 Hz, treble cut at 8 kHz):

- Shannon wavelet scale j corresponds to octave band $[2^j\pi, 2^{j+1}\pi]$.
- To boost scale j by +6 dB: multiply all $d_{j,k}$ by 2.
- To apply a shelf filter: ramp the gain across scales.

Advantage over IIR/FIR EQ:

- IIR (biquad cascade): 2nd-order sections with stability concerns, phase distortion.
- FIR (Parks-McClellan): linear phase but high order ($\sim 100+$ taps for sharp transitions).
- Shannon wavelet: natural octave bands, perfect reconstruction, no phase distortion within each band.

Quantization Error in Synthesis Context

For a signal with n scales, total synthesis error is:

$$\mathcal{E}_{\text{total}} = \underbrace{\sum_k |c_{j_0,k}|^2 \mathcal{E}_{\text{quant}}^{(j_0)}}_{\text{base scale}} + \underbrace{\sum_{j=j_0}^{j_0+n-1} \sum_k |d_{j,k}|^2 \mathcal{E}_{\text{quant}}^{(j)}}_{\text{detail scales}},$$

where $\mathcal{E}_{\text{quant}}^{(j)} = \sum_m S_{\psi,m}^{(j)} / 2^{m_{\psi,m}^{(j)}+1}$ from Section 4.4.6.

Key insight for synthesis:

- High frequencies (large j): typically have *small* coefficients $|d_{j,k}|$ (natural signal decay).
- Low frequencies (small j): have *large* coefficients but fewer wavelets needed.
- The bit allocation vector $\mathbf{m}$ can assign *fewer bits to high-frequency scales* where $|d_{j,k}| \ll 1$, saving hardware resources without audible/visible degradation.

Practical synthesis example:

- 5-scale decomposition for audio (20 Hz – 20 kHz).
- Scale 0 (j_0): 20–40 Hz (bass), $m_n = 16$ bits (high precision for low distortion).
- Scale 1: 40–80 Hz, $m_n = 14$ bits.
- Scale 2: 80–160 Hz, $m_n = 12$ bits.
- Scale 3: 160–320 Hz, $m_n = 10$ bits.
- Scale 4: 320–640 Hz and above, $m_n = 8$ bits (high frequencies masked by louder low frequencies).
- **Total bits:** $16 + 14 + 12 + 10 + 8 = 60$ bits distributed across 5 scales.
- **Uniform quantization:** would use $12 \times 5 = 60$ bits but with *worse* perceptual quality (over-quantizes bass, under-quantizes treble relative to perception).

Comparison: Shannon Wavelets vs. Alternatives

Method	Freq. Selectivity	Time Localization	Hardware Cost	Phase
Shannon Wavelet	Excellent	Poor	Low (dyadic ops)	Linear
Haar Wavelet	Poor	Excellent	Very Low	Linear
Daubechies Wavelet	Good	Good	Medium	Nonlinear
FFT/STFT	Excellent	Poor	Medium	Complex
FIR Filter	Good	Medium	High (N taps)	Linear
IIR Filter	Good	Good	Low	Nonlinear
Polynomial (dyadic)	N/A	Excellent	Very Low	N/A

Table A.1: Comparison of signal synthesis methods for bandlimited signals.

When to use Shannon wavelets:

- **Frequency-domain processing** with minimal computational cost (filtering, EQ, pitch shift).
- **Multi-rate systems** where different frequency bands are processed at different rates (e.g., low frequencies at 1 kHz, high frequencies at 48 kHz).
- **Adaptive quantization** where bit budget must be allocated across frequency bands based on sensitivity.
- **Perfect reconstruction** is required (lossless analysis-synthesis).

When NOT to use Shannon wavelets:

- **Transient-heavy signals** (percussion, clicks) — use Daubechies or Haar for better time localization.
- **Wideband noise** — time-domain polynomial or direct sampling is simpler.
- **Ultra-low latency** — Shannon wavelets have infinite time support (though truncated in practice).

Synthesis Quality Metrics

For the quantized Shannon wavelet synthesis $\tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')} (t)$ from (4.60):

Signal-to-Quantization-Noise Ratio (SQNR):

$$\text{SQNR} = 10 \log_{10} \frac{\|f\|_{L^2}^2}{\|f - \tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')} \|_{L^2}^2} \quad (\text{dB}).$$

For the greedy bit allocation (Appendix A.3) with total budget $B_{\text{total}} = 60$ bits across 5 scales:

- Typical SQNR: 70–90 dB (depending on signal).
- Comparable to 12-bit uniform quantization on the time-domain signal but with *perceptually optimized* distribution.

Total Harmonic Distortion (THD): For synthesis of harmonic signals (musical tones):

$$\text{THD} = \frac{\sqrt{\sum_{k=2}^{\infty} P_k}}{P_1} \times 100\%,$$

where P_k is the power in the k -th harmonic.

With $N = 4$ polynomial terms per wavelet and $m_n \in [8, 16]$ bits:

- $\text{THD} < 0.01\%$ for fundamental + first 8 harmonics (professional audio quality).
- $\text{THD} < 0.1\%$ for 32 harmonics (CD quality).

Bandwidth efficiency:

- Shannon wavelets are *optimal* for bandlimited signals in the sense that the number of coefficients $\{c_{j_0,k}, d_{j,k}\}$ equals the number of degrees of freedom (Shannon's sampling theorem).
- For a signal bandlimited to B Hz sampled at $f_s = 2B$, the Shannon wavelet decomposition requires $\sim f_s$ coefficients per second, which is *information-theoretically optimal*.